



CENG 408

Innovative System Design and Development

2

2025-2026 Spring

Project:

**Syntagma: AI Powered Spaced Repetition System
Based Language Learning App**

Team Members:

202211403 İlker Feza Çoban

202111027 İlayda Sümer

202111007 Elif Su Tufan

202211077 Alp Yılmaz

202211032 Sultan Gürbüz

202111417 Ömerhan Yiğit

Advisor: Assist. Prof. Dr. Abdül Kadir Görür

Table of Contents

Table of Figures:.....	9
Abstract	10
Introduction	11
Section 1: Literature Research	12
1. Existing Works:	12
1.1 Existing Language Learning Applications.....	12
1.2 Foreign Language Acquisition Theories.....	12
1.3 Spaced Repetition System (SRS)	13
1.3.1 SM-2 (SuperMemo-2)	13
1.3.2 FSRS	14
2.System Architecture Components:	15
2.1 Backend System:	15
2.1.1 Gradle	15
2.1.2 Hibernate + Spring Data JPA	15
2.1.3 Lombok	16
2.1.4 Mockito	16
2.1.5 Spring Boot:.....	16
2.1.6 SLF4J:.....	17
2.1.7 Logback:.....	17
2.1.8 PostgreSQL:.....	18
2.1.9 DBeaver:.....	18
2.1.10 Docker:	19
2.1.11 Swagger:	19
2.1.12 Postman:	19
2.2 Chrome Extension with NLP-Based Flashcard Generation	20
2.2.1 Subtitle and Text Extraction	20
2.2.2 Dictionary Lookup and Interaction	20
2.2.3 NLP-Driven Flashcard Creation	20

2.3	Frontend and Design.....	21
2.3.1.	User Interface (UI) Design and User Experience (UX)	21
2.3.2	Gamification and Motivation	22
2.3.3	Personalization and Adaptive Learning	23
2.3.5.	Push Notification Frequency and User Behavior	25
2.3.6.	React Native in Mobile App	26
Section 2: Product Vision & Scope: Syntagma.....		27
1.	Product Vision.....	27
2.	Scope Boundaries	27
2.1	In-Scope Features	27
2.2	Out of scope:	28
3.	Feature Breakdown	28
3.1	Chrome Extension	28
3.2	NLP Engine.....	29
3.2.1	Text Processing.....	29
3.2.2	Language Configuration Layer.....	29
3.3	Backend.....	29
3.3.1.	APIs.....	29
3.3.2.	Database	30
3.3.3.	Spaced Repetition System.....	31
3.3.4.	Logging	31
3.4	Mobile Application:	31
3.4.1.	SRS Review Experience.....	31
3.4.2.	Flashcard Viewer	31
3.4.3.	Dashboard.....	31
3.4.4.	Meaning / Dictionary Integration	32
3.4.5.	Gamification.....	32
3.4.6	Push Notifications.....	32
3.4.7.	Offline Review Support	32

4. User Stories:	32
Section 3: System Architecture & Design Decisions	33
1. Functional Requirements.....	33
Chrome Extension.....	33
NLP	40
Backend	41
Mobile Application.....	43
2. Use Case Diagrams:	46
Chrome Extension:	46
Mobile Application:.....	47
3. Data Flow Diagrams:	48
Section 4: Implementation Details	49
1. Introduction.....	49
2. Backend Application	49
2.1 Technology Stack	49
2.2 Architectural Structure	50
2.3 Core Business Modules.....	50
2.3.1 Advanced Spaced Repetition (FSRS Algorithm)	50
2.3.2 Flashcards & Collections.....	50
2.3.3 Sync and Offline Support	50
2.3.4 AI-Assisted Learning	50
2.3.5 Media and Ebook Management	50
2.3.6 Word Knowledge Tracking	51
2.4 Security & Exception Handling	51
2.5 Configuration Highlights.....	51
3. Chrome Extension.....	51
3.1 Purpose and Scope.....	51
3.2 System Context.....	51
3.3 Technology Stack	52

3.4 Runtime Architecture.....	52
3.4.1 Background Layer	52
3.4.2 Shared Layer.....	52
3.4.3 Content Layer	53
3.4.4 Extension Pages Layer	53
3.5 Core Data Model	53
3.5.1 User settings	53
3.5.2 Lexeme state	54
3.5.3 Flashcards.....	54
3.5.4 Dictionary store	54
3.5.5 Auxiliary storage keys	54
3.6 Message Contract and Communication	54
3.7 Page Parsing and Overlay Pipeline.....	55
3.7.1 Parsing flow	55
3.7.2 Overlay injection	55
3.7.3 User interaction logic	55
3.8 Word Popup and Learning Actions	55
3.9 Video Subsystem.....	56
3.9.1 Mounting strategy	56
3.9.2 Subtitle capture sources	56
3.9.3 Synchronization model.....	56
3.9.4 Interaction capabilities.....	56
3.9.5 Built-in Video Player Page.....	57
3.9.6 Audio Capture for Flashcards.....	57
3.10 Integrations	57
3.10.1 Dictionary audio API.....	57
3.10.2 Backend sync.....	57
3.10.3 AI Example Sentence Generation	57
3.11 Storage and Persistence Summary	58

3.12 Testing and Quality Status	58
3.13 Operational Notes and Known Limitations	58
3.14 Extension Pages	58
3.14.1 Workspace (card-creator.html)	58
3.14.2 Auth Page (auth.html)	58
3.14.3 Settings Page (options.html).....	59
3.14.4 eBook Reader Page (reader.html)	59
3.14.5 Popup Page (popup.html)	59
3.15 Internationalization (i18n) System.....	59
3.16 Contraction and Morphological Handling.....	59
3.17 CEFR Level Support.....	59
4. Mobile Application.....	60
4.1 Purpose and Scope.....	60
4.2 Architecture	60
4.3 Data Model	60
4.4 Synchronization and Offline Strategy.....	60
4.5 Security	61
4.6 Testing and Quality.....	61
Section 5: Testing and Validation.....	61
1. Introduction	61
1.1 Version Control	61
1.2 Overview.....	61
1.3 Scope	62
1.4 Terminology	62
2. Features to be Tested.....	62
2.1 Authentication and Account Management (AUTH)	62
2.2 Content Parsing and Comprehension Analysis (CPA)	62
2.3 Flashcard Creation and Collections (FCC)	62
2.4 Video Subtitle and Media Capture (VSM)	63

2.5 Spaced Repetition Review and Mobile Learning (SRS)	63
3. Features Not to be Tested	63
4. Item Pass/Fail Criteria	63
4.1 Exit Criteria	63
5. References	63
6. Test Design Specifications	64
6.1 Authentication and Account Management (AUTH)	64
6.1.1 Subfeatures to be tested	64
6.1.2 Requirements	64
6.1.3 Test Cases	64
6.2 Content Parsing and Comprehension Analysis (CPA)	65
6.2.1 Sub features to be tested	65
6.2.2 Requirements	65
6.2.3 Test Cases	65
6.3 Flashcard Creation and Collections (FCC)	66
6.3.1 Subfeatures to be tested	66
6.3.2 Requirements	66
6.3.3 Test Cases	66
6.4 Video Subtitle and Media Capture (VSM)	67
6.4.1 Subfeatures to be tested	67
6.4.2 Requirements	67
6.4.3 Test Cases	67
6.5 Spaced Repetition Review and Mobile Learning (SRS)	68
6.5.1 Subfeatures to be tested	68
6.5.2 Requirements	68
6.5.3 Test Cases	68
7. Detailed Test Cases	69
7.1 AUTH.REG.01	69
7.2 AUTH.LOG.01	69

7.3 AUTH.JWT.01	70
7.4 AUTH.PRO.01	70
7.5 CPA.TOK.01	71
7.6 CPA.LEM.01	71
7.8 CPA.SCR.01	72
7.9 CPA.REA.01	73
7.10 FCC.POP.01	74
7.11 FCC.AL.01	74
7.12 FCC.CRT.01	75
7.13 FCC.EDT.01	75
7.15 FCC.MAP.01	76
7.16 VSM.YT.01	77
7.17 VSM.FB.01	77
7.18 VSM.DSP.01	78
7.19 VSM.SS.01	78
7.20 VSM.AUD.01	79
7.21 VSM.MEDIA.01	79
7.22 SRS.DUE.01	80
7.23 SRS.NEW.01	80
7.24 SRS.LAP.01	81
7.25 SRS.REV.01	82
7.26 SRS.STA.01	82
7.27 SRS.MOB.01	83
Section 6: Conclusion and Future Work	83
1. Project Summary	83
2.1 FSRS-Based Spaced Repetition Engine	84
2.2 AI-Powered Language Assistance	84
2.3 Chrome Extension Learning Features	84
2.4 Mobile Application	85

2.5 Robust Backend Architecture	85
2.6 Word Knowledge Tracking	86
3. Limitations	86
4. Lessons Learned	86
5. Conclusion, Shortcomings, Future Works	87
5.1. Backend Application	87
5.1.1 Conclusion	87
5.1.2 Shortcomings	87
5.1.3 Future Works	88
5.2.1 Conclusion	88
5.2.2 Shortcomings	88
5.2.3 Future Works	88
5.3 Mobile Application	88
5.3.1 Conclusion	88
5.3.2 Shortcomings	88
5.3.3 Future Works	89
5.4.1 Conclusion	89
5.4.2 Shortcomings	89
5.4.3 Future Works	89
References	89

Table of Figures:

Figure 1 SM-2	14
Figure 2 Spring Boot working diagram	17
Figure 3 Use Case Diagram of Chrome Extension	46
Figure 4 Use Case Diagram of Mobile Application	47
Figure 5 Data Flow Diagram of Syntagma Learning System Level 0	48
Figure 6 Data Flow Diagram of Syntagma Learning System Level 1	48

Abstract

Syntagma is a smart language learning app that uses artificial intelligence (AI) to help people learn foreign languages more effectively. The project's main goal is to turn regular media consumption, like watching videos or reading online, into an effective way to learn vocabulary and achieve fluency.

Our philosophy is based on the idea that people learn languages best through "comprehensible input"—meaning content they understand, but which contains just enough new information to push them forward. Therefore, Syntagma avoids traditional grammar drills and textbooks, focusing instead on fun, interesting media to keep users engaged and reduce learning stress.

To ensure users remember what they learn, the app uses a system called Spaced Repetition (SRS). This system schedules reviews for words at the perfect moment—right before the user would normally forget them—to maximize memory retention while minimizing study time. We use a modern, advanced algorithm called FSRS, which uses machine learning to analyze the user's memory patterns and predict the ideal time to review each flashcard.

The application has two main parts:

1. The Server (Backend): This robust system is built using reliable tools like Spring Boot and PostgreSQL (a high-performance database). The whole setup is packaged using Docker to ensure the app works consistently everywhere.
2. The User Interface and Content Tool: A Chrome Extension is used to extract subtitles and text from websites and video platforms. This extracted text is processed using Natural Language Processing (NLP) tools that automatically identify important words, filter out irrelevant content, and create high-quality, context-aware flashcards.

The mobile app is designed to be highly motivating. It includes gamification features like points, streaks, and badges to encourage users to practice daily and build learning habits. The system also uses personalization to adapt content and review schedules to the individual learner's progress. By combining media consumption with efficient, data-driven memory training, Syntagma delivers a personalized and engaging path toward language fluency.

Introduction

The Syntagma project introduces a new, smart application designed to make learning foreign languages easier and more effective. Many popular learning platforms, like Duolingo, focus on making lessons fun games, but they often do not present the deep, contextual study needed for fluency. Other powerful tools, such as the combination of Yomitan and Anki, are highly effective but are difficult and time-consuming for most learners to set up and maintain. Syntagma aims to solve this gap by bringing together the fun parts of learning with the efficiency of advanced study techniques. Our approach is built on the educational framework known as Stephen Krashen's Monitor Model. This theory suggests that fluency comes from natural acquisition (like a child learning their first language) rather than just the conscious study of grammar rules. Our main goal is to reduce stress (the "affective filter") and allow users to learn from content they genuinely enjoy, such as videos or dramas. To make this learning system efficient, Syntagma focuses on providing comprehensible input ($i + 1$)—content that is mostly understood but contains just enough new information to drive progress. We achieve this through two main technical components. First, a Chrome Extension is used to extract subtitles and text directly from external media sources. This raw text is then processed using smart tools, known as Natural Language Processing (NLP), to automatically create high-quality, contextual flashcards. Second, to ensure that vocabulary is remembered long-term, the system uses an advanced Spaced Repetition System (SRS). This system employs the FSRS algorithm, which uses machine learning to accurately predict a user's memory state and schedule reviews at the exact moment needed to maximize retention while minimizing study time. Finally, the mobile application uses gamification elements, like streaks and points, along with personalization, to keep users motivated and help them build strong, consistent learning habits.

Section 1: Literature Research

1. Existing Works:

1.1 Existing Language Learning Applications

These are the tools that are widely used for foreign language acquisition.

Gamified Platforms (Duolingo [1], Memrise [2]):

Duolingo is a market leader in terms of user numbers, targeting casual beginners with short, game-like lessons, points, streaks, etc. Their approach is fundamentally different from Syntagma's, prioritizing engagement and habit formation through extrinsic motivators over deep, contextual learning.

Curriculum-Based Platforms (Babbel [3], Busuu [4], Rosetta Stone [5]):

These applications provide more structured, course-like experiences that are closer to traditional classroom learning.

Language Reactor:

Operating as a freemium browser extension or a website with integrated content from sites such as YouTube or Netflix. However, there are no way to upload your own content, or study those materials effectively.

LingQ:

LingQ is built around a reading-centric approach [6]. It boasts an extensive library of texts with synchronized audio and tracks vocabulary progress. While its methodology is very similar to Syntagma's, it's focus is more on text.

Yomitan + Anki:

For technically proficient learners, these combinations of free, open-source tools are free to use. Yomitan serves as a pop-up dictionary extension, and Anki is used for spaced repetition [7]. These tools can replicate much of Syntagma's core functionality however these tools are very hard to set up and can cost a lot of time for language learners.

1.2 Foreign Language Acquisition Theories

The input hypothesis and $i + 1$:

The philosophical foundation of Syntagma is a direct application of Stephen Krashen's monitor model, particularly his influential input Hypothesis. This theory has several claims that are included in the application's design.

The Acquisition-Learning Distinction:

Krashen distinguishes between “acquisition” which is a subconscious process similar how a child learns their first language and “learning”, the conscious study of grammar rules. He hypothesizes that fluency comes from acquisition, not learning. Syntagma’s rejection of traditional textbooks and grammar drills in favor of fun, interesting media consumption is an embodiment of this principle.

Comprehensible Input ($i + 1$) :

The core of this hypothesis is that “acquisition” occurs when learners are exposed to “comprehensible input” that is just one step beyond their current level of understanding, previously stated as ($i + 1$).

The input must be understood but also contain new elements to drive progress. Syntagma implements this concept directly by tracking the user’s vocabulary and making a “comprehension score” of each piece of media. The suite of dictionary, translation, AI explanation tools also serve to make difficult ($i + 2$, $i + 3$ etc) sentences into more easily digestible ($i + 1$) sentences.

The Affective Filter Hypothesis:

Krashen also proposed that emotional factors like anxiety, stress, and low motivation can create an “affective filter” that blocks comprehensible input from being processed.

Syntagma’s emphasis on allowing users to learn from content they genuinely enjoy such as YouTube videos, dramas, anime is a practical strategy for lowering this filter and making the learning process engaging.

1.3 Spaced Repetition System (SRS)

Spaced repetition systems (SRS) schedule reviews at expanding intervals to exploit the spacing effect and the forgetting curve: easier items are scheduled less often, harder items more often, so a learner maintains a target recall level while minimising reviews.

1.3.1 SM-2 (SuperMemo-2)

SM-2 is a lightweight algorithm originally used in SuperMemo that updates a card’s easiness factor and interval based on the user’s grade for each review.

Key variables (per card):

EF = easiness factor (initial 2.5 by default)

- $I[n]$ = interval after n th successful repetition (in days)
- n = repetition count (consecutive successful reviews)
- q = quality response from user, typically 0–5 (in Anki it's mapped to 4 choices)

Core SM-2 rules (text):

1. If $q < 3$ (failure), set $n = 0$, schedule the card for immediate/short repeat (e.g., 0 or 1 day).
2. If $q \geq 3$:
 - a. If $n = 0 \rightarrow I[1] = 1$
 - b. If $n = 1 \rightarrow I[2] = 6$
 - c. If $n \geq 2 \rightarrow I[n+1] = I[n] * EF$
 - d. Update EF: $EF := EF + (0.1 - (5 - q) * (0.08 + (5 - q) * 0.02))$
 - e. If $EF < 1.3$ then $EF := 1.3$ (lower bound)

1.3.2 FSRS

FSRS aims to learn your memory patterns and schedule reviews more efficiently than SM-2.

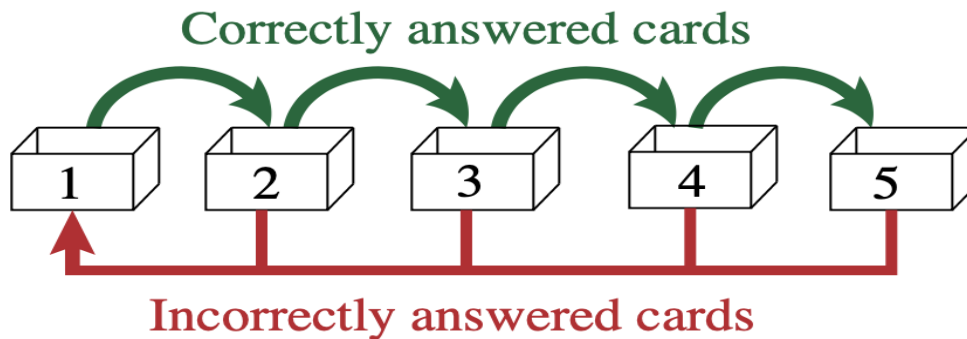


Figure 1 SM-2

FSRS is based on the Three Component Model of Memory [8]. The model asserts that three variables are sufficient to describe the status of a unitary memory in a human brain. These three variables include:

- **Retrievability (R):** The probability that the person can successfully recall a particular information at a given moment. It depends on the time elapsed since the last review and the memory stability (S).
- **Stability (S):** The time, in days, required for R to decrease from 100% to 90%. For example, $S = 365$ means that an entire year will pass before the probability of recalling a particular card drops to 90%.

- Difficulty (D): The inherent complexity of a particular information. It represents how difficult it is to increase memory stability after a review.

In FSRS, these three values are collectively called the "memory state". The value of R changes daily, while D and S change only after a card has been reviewed. Each card has its own DSR values, in other words, each card has its own memory state. To accurately estimate the DSR values, FSRS analyzes the user's review history and uses machine learning to find parameters that provide the best fit to the review history.

2.System Architecture Components:

2.1 Backend System:

2.1.1 Gradle

Gradle is a build automation tool used to compile, test, and package Java projects (like Spring Boot applications) [9].

It manages dependencies — the external libraries your project needs — and automates repetitive tasks such as building .jar files, running tests, or deploying the app.

Gradle uses Groovy or Kotlin scripts instead of XML (like Maven), which makes it faster and more flexible.

It also supports incremental builds, meaning it only rebuilds the parts of the project that changed, saving time.

In this project, Gradle is used for:

- Manage dependencies (Spring Boot, Hibernate, Lombok, etc.)
- Build and run the backend application

2.1.2 Hibernate + Spring Data JPA

Hibernate is an Object-Relational Mapping (ORM) tool that connects Java objects to relational database tables [10].

It automatically generates SQL queries from your Java code, so you don't need to write raw SQL for CRUD operations.

Spring Data JPA builds on Hibernate to simplify database operations even further.

By defining repository interfaces, developers can perform complex queries and manage entities without writing SQL or boilerplate code.

In this project, Hibernate is used for:

- Eliminates manual SQL query writing
- Reduces boilerplate code
- Manages entity relationships and transactions automatically

2.1.3 Lombok

Lombok is a Java library that reduces boilerplate code [11] by automatically generating commonly used methods such as getters, setters, constructors, toString(), and equals/hashCode().

It integrates with IDEs and the build system, allowing to write cleaner and shorter code while maintaining full functionality [12].

In this project, Lombok is used for:

- Eliminates repetitive code for entity classes and other Java objects
- Simplifies maintenance and improves readability
- Speeds up development by reducing manual coding of boilerplate methods [11]

2.1.4 Mockito

Mockito is a popular Java library used for unit testing by creating mock objects [13].

Mocking allows to simulate the behavior of external dependencies such as databases, APIs, or services, so that tests can focus only on the component being tested [14].

In this project, Mockito is used for:

- Eliminates the need to access real databases or external services during testing
- Makes unit tests faster, more reliable, and isolated
- Allows developers to control and verify interactions with dependencies

2.1.5 Spring Boot:

Spring Boot is a sub-framework derived from the Spring ecosystem. In the modern trend of separating the front-end and back-end layers, Spring Boot allows developers to build server-side applications quickly and efficiently. Its auto-configuration mechanism significantly reduces the need for manual setup and enhances development productivity [15]

Traditional Spring projects often required large and complex configuration files, but Spring Boot eliminates this problem by introducing an automated and lightweight configuration process. Through Spring Boot starters, the framework automatically manages the necessary dependencies for the

application. With the help of build automation tools such as Maven or Gradle, developers can easily add essential modules, including web and data access components. This approach enables them to focus on implementing business functionality rather than spending time on extensive configuration tasks.

Spring Boot's microservices architecture enhances the scalability and maintainability of these applications, allowing for the seamless addition of microservices to meet evolving service requirements.

Spring Boot's role in creating a Service-Oriented Architecture (SOA)-based REST service API at the Atmospheric Radiation Measurement (ARM) Data Center showcases its utility in bridging the divide between frontend user interfaces and backend databases [16]

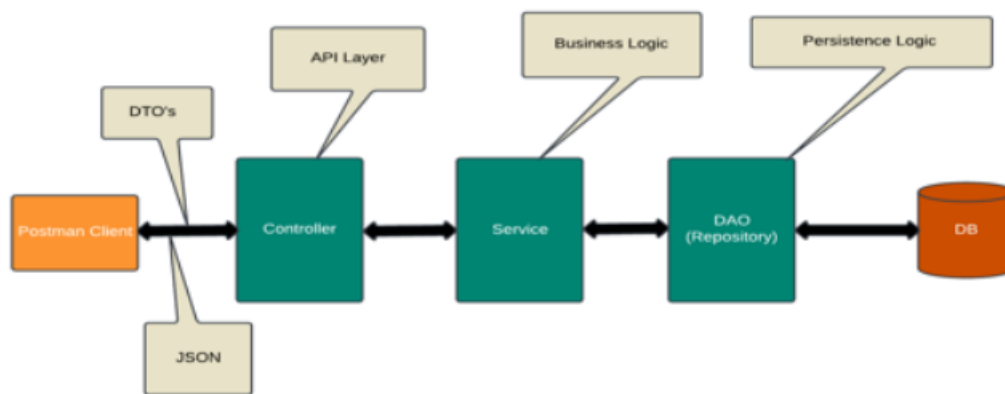


Figure 2 Spring Boot working diagram

2.1.6 SLF4J:

The Simple Logging Facade [17] for Java, serves as a simple facade or abstraction for various logging frameworks. SLF4J allows the end-user to plug in the desired logging framework at deployment time [18].

In this project, SLF4J is used for:

- SLF4J is facade and we can write code once but switch to another logging framework without changing any line of code.
- Reduces coupling [19] between application and logging framework

2.1.7 Logback:

Logback is a logging framework that works perfectly with Spring-Boot applications [20].

In this project, Logback is used for:

- Built-in support for SLF4J

2.1.8 PostgreSQL:

Several studies have compared relational databases in the context of e-learning and high-performance data management applications, often highlighting the advantages of PostgreSQL over conventional systems like MySQL.

Jeyaraj et al. (2022) proposed a system designed to promote the adoption of PostgreSQL by demonstrating its strengths and advantages compared to other relational databases, particularly MySQL [21]. The authors highlight that PostgreSQL, as an object-relational database management system (ORDBMS), offers advanced SQL compliance and superior performance capabilities. However, they also noted that its complex textual interface and lack of beginner-friendly tools limit its widespread use. To address this, their study developed a visual and interactive learning platform that helps users understand PostgreSQL's structure, query manipulation, and relational modeling. The research emphasizes PostgreSQL's potential as the most advanced open-source RDBMS and its value for educational and industrial database systems.

Further substantiating this position, Salunke and Ouda (2024) conducted a comprehensive benchmarking study comparing the performance of PostgreSQL and MySQL under real-world, high-load scenarios. Their research introduced a Python-based benchmarking framework to evaluate query execution time, latency, and concurrent processing capabilities. Experimental results demonstrated that PostgreSQL achieved significantly lower latency than MySQL—up to faster in select queries and faster in conditional selects—while maintaining stable performance during simultaneous operations [22]. These findings confirm PostgreSQL's suitability for applications requiring low data latency and high concurrency, making it an optimal choice for large-scale backend systems such as learning platforms or continuous authentication environments. This collective body of research indicates a growing trend toward adopting PostgreSQL for scalable and educational database systems due to its superior architectural and performance advantages.

2.1.9 DBeaver:

DBeaver is a universal database management tool. DBeaver allows for easy data editing in a spreadsheet-like interface [23].

DBeaver was chosen for the Spring Boot-based development process to visualize and manage the database structure more efficiently. This tool significantly simplifies the processes of reviewing, editing, and managing database designs within the project.

2.1.10 Docker:

Mhatre (2023) examined the role of Docker and Kubernetes within microservices-based software architectures, emphasizing their contributions to scalability, flexibility, and reliability in modern development. [24] Docker ensures consistent runtime environments across development, testing, and production stages, eliminating deployment discrepancies, while Kubernetes provides automated orchestration, scaling, and load balancing for containerized applications.

In this project, however, Docker was preferred over Kubernetes because, although Kubernetes offers powerful orchestration capabilities for large-scale distributed systems, it introduces higher complexity, greater resource consumption, and longer setup time for small to medium-sized projects. Docker provides a simpler configuration, faster deployment, and sufficient scalability for the project's backend and database services. Moreover, using Docker Compose, multiple services (e.g., Spring Boot backend and PostgreSQL database) can be easily linked within the same container network.

Thus, Docker was selected as the primary containerization technology to simplify deployment, ensure portability, and improve development efficiency.

2.1.11 Swagger:

Lama (2025) emphasizes in his study that API documentation is a critical component in software projects that directly impacts interteam communication and development efficiency.

He stated that traditional, static documentation methods lead to problems such as outdatedness, inconsistency, and lack of interaction, which in turn prolongs development time and leads to integration errors.

The study recommends the automated, standardized, and interactive documentation structure provided by Swagger to address these issues.

Swagger integrates with Java-based frameworks such as Spring Boot, automatically keeping API endpoints, request/response models, and data schemas up-to-date.

This approach both streamlines the flow of information between developers and enables new team members to quickly adapt to the system.

The study's results demonstrate that Swagger provides transparency, accuracy, and speed in API management and forms the foundation of interactive API ecosystems in modern software development [25].

2.1.12 Postman:

Kore et al. (2022) presented an overview of Postman, a leading platform for API testing and automation widely adopted by developers and enterprises [26]

The study outlines how traditional manual API testing methods fail to scale in modern software ecosystems where applications interact with hundreds of APIs simultaneously.

Postman simplifies this process by providing a graphical interface for designing, executing, and validating RESTful API requests (GET, POST, PUT, DELETE) while offering integrated support for headers, parameters, and authentication.

Additionally, the paper discusses Newman, Postman's CI/CD add-on, which enables automated and continuous testing as part of DevOps pipelines.

Their implementation in a banking and financial services project demonstrated how Postman enhances collaboration, efficiency, and test repeatability, making it a valuable tool for verifying API reliability and security.

2.2 Chrome Extension with NLP-Based Flashcard Generation

Since most of our media consumption is done on browsers, it makes sense to embed our learning tools directly into the user's web experience. The Chrome extension acts as the gateway between web content and Syntagma's personalized language learning system.

2.2.1 Subtitle and Text Extraction

The extension will interface with video platforms and general websites to extract readable text content.

Technologies can be used:

- yt-dlp: A command-line tool to download videos and subtitles from platforms like YouTube [27].
- netflix-to-srt: A GitHub tool that allows subtitle extraction from Netflix videos [28].
- Readability.js: Mozilla's tool for cleaning articles by stripping cluttered HTML from web pages [29]

2.2.2 Dictionary Lookup and Interaction

Users can click or hover on words in subtitles or website text. The extension displays a pop-up with dictionary definitions, translations, and example usage.

2.2.3 NLP-Driven Flashcard Creation

This stage transforms selected or parsed sentences into high-quality flashcards using multiple NLP components:

a. Sentence Segmentation and Tokenization

- `wink-nlp`: A fast JavaScript NLP library supporting sentence segmentation and word tokenization [30].
- `compromise.js`: Provides tokenization, lemmatization, and POS tagging for browser environments [31].

b. Lemmatization and POS Tagging

These help reduce words to their root forms and identify their grammatical roles.

c. Frequency and Difficulty Filtering

SUBTLEX word lists will be used to filter low-frequency or less relevant vocabulary [32].

e. Named Entity Recognition (NER)

`spaCy` [33] (for Python) and `compromise.js` (for JavaScript) to filter out named entities that don't require flashcard creation.

f. Audio Extraction and TTS Generation

Native audio can be extracted using `yt-dlp`.

If unavailable, synthetic audio can be generated using Google TTS [34].

2.3 Frontend and Design

2.3.1. User Interface (UI) Design and User Experience (UX)

The front-end design of mobile language learning applications is one of the most critical components directly affecting the user experience. The User Interface (UI) in these applications is not merely a visual element but a key factor that determines the effectiveness of the learning process [35].

Research on leading applications such as Duolingo, Memrise, Babbel, Busuu, and LingQ shows that all of them commonly adopt a philosophy of simple, user-friendly, and engagement-oriented design. The interfaces of these applications are mostly button-based, aiming to allow users to act quickly and intuitively during the learning process. Menu structures are intentionally kept minimal and guiding, thereby avoiding unnecessary distractions.

Each application establishes a unique identity through its distinctive color palette and iconography. This consistent visual theme strengthens user attachment to the application and reinforces visual coherence.

UI/UX design principles focus on avoiding complex transitions and information overload. Instead, designers prioritize visual elements that support learning:

- Clear Icons: Symbols whose meaning is immediately comprehensible.
- Consistent Typography: Font styles that enhance readability and reflect brand identity.
- Intuitive Navigation: Components that enable users to easily reach their desired location.

Furthermore, instant feedback mechanisms play a vital role in the user experience. Users immediately see whether their answers are correct or incorrect, reinforcing learning through immediate response and repetition. This fast feedback loop creates an interactive and supportive learning environment [35].

2.3.2 Gamification and Motivation

Gamification is one of the most widely used strategies to enhance user motivation in mobile language learning applications. This approach integrates game-like elements into the learning process.

Key Gamification Elements:

- Rewards and Points (XP): Ensures users see a tangible return for their efforts.
- Badges and Achievements: Creates a sense of accomplishment upon reaching specific milestones.
- Levels and Streaks: Establishes continuous progression and a habit of regular practice.

These elements increase both the frequency of engagement and the retention of acquired knowledge [36]. Through gamification, users tend to open the app more frequently, set specific goals, and feel a sense of accomplishment upon completing them. These mechanisms provide immediate motivation in the short term, while aiding in the construction of sustainable learning habits over the long term.

Applications like Duolingo [1], Memrise [2], and Busuu [4] extensively apply these elements. For example, in Duolingo, users who practice daily without skipping sessions earn "streak" points or receive badges for completing milestones. These systems not only make learning enjoyable but also foster a positive emotional connection between the user and the app.

Significantly, gamification encourages voluntary engagement rather than forced participation. Studies have shown that voluntary users interact with language learning applications more frequently and for longer durations compared to mandatory users. This highlights that the inclusion of motivation-enhancing features is critical for maintaining long-term user engagement and consistent application use [36].

2.3.3 Personalization and Adaptive Learning

Personalization is another defining feature of modern mobile language learning applications. These systems analyze user data—such as learning habits, progress pace, and areas of strength or weakness—to create individualized recommendations, reminders, and review schedules [37].

This adaptive design allows each learner to control their own learning journey. For instance, LingQ enables learners to progress at their own pace while providing relevant vocabulary and content recommendations tailored to their interests and current proficiency level. This makes the learning experience more relevant and efficient [37].

Push Notifications also play a critical role in sustaining user engagement [38]. Regularly sent reminders encourage users to return to the app, helping maintain consistency in their learning routine. These features not only enhance user commitment but also help prevent interruptions in the learning process [37].

2.3.4. Accessibility and Mobile Learning Advantages

Mobile language learning apps make education flexible, personalized, and available anywhere [37]. Unlike traditional classroom settings, Mobile-Assisted Learning allows users to study anytime using their smartphones or tablets. This is ideal, especially for individuals with limited time or restricted access to formal education [37].

A key benefit of mobile accessibility is ubiquitous access. Users can find learning opportunities even during short daily breaks, such as commuting. Apps like Duolingo and Memrise offer short "micro-lessons" that fit easily into busy routines. Furthermore, many applications support an offline mode, increasing inclusivity for users in areas with poor internet connectivity [37].

Finally, multimodal learning and push notifications sustain engagement. Combining text, audio, and visuals strengthens vocabulary and pronunciation [39], while reminders and progress tracking encourage consistent study habits [37]. Overall, mobile accessibility enables a flexible, inclusive, and continuous approach to language learning.

Comparison Table of Major Language Learning Apps:

Application	UI/UX Design	Gamification Level	Personalization & Adaptive Learning	Notification Usage	Vocabulary / Word Management	Additional Notes
Duolingo	Colorful, minimal, icon-based; fast tap-interaction	Very high: streaks, XP, badges, levels	Moderate: adjusts lesson difficulty based on progress	Frequent daily reminders	Basic vocabulary tracking; limited customization	Highly gamified; widely used; simple micro-lessons
Memrise	Visual-focused; simple layout; includes user-generated video clips	Medium: points, levels	Low-medium: adjusts pace slightly	Frequent motivational reminders	Strong flashcard system; users can create their own decks	Uses real native-speaker videos; community-driven content
Babbel	Clean, serious, professional UI; lesson-focused	Low: minimal gamification	Strong: structured lesson paths based on level	Moderate notifications	Strong review system; spaced repetition	More academically structured; requires subscription
Busuu	Modern design; strong community integration	Medium: achievements and goals	High: personalized study plan + AI reviews	Regular reminders	Personal vocabulary list; community corrections	Social learning: natives correct your writing/speaking
LingQ	Text-heavy, simple UI; color-coded words	Very low	Very high: personalized content, smart suggestions, vocabulary-based adaptation	Moderate	One of the most advanced vocab systems (highlighting, stats, word tracking)	Best for reading & listening immersion; highly customizable

2.3.5. Push Notification Frequency and User Behavior

Push notifications play a central role in sustaining user engagement in mobile learning environments by prompting users to return to the application and maintain consistent study habits. However, notification frequency is a decisive factor that can significantly influence user behavior, perceived app intrusiveness, and long-term retention. The empirical findings of Wohllebe et al. (2021) provide one of the most comprehensive experimental analyses on this topic, making it directly relevant for mobile-based language learning applications [40].

In their large-scale experiment involving 17,500 real app users, five different notification frequency groups were observed over a seven-week period. The results demonstrated that higher notification frequency leads to a substantial increase in uninstall rates. Specifically, users receiving two notifications per week exhibited noticeably higher uninstall percentages compared to groups receiving weekly or bi-weekly notifications [40]. This indicates that excessive notifications can be perceived as intrusive, ultimately harming user retention.

Regarding user interaction patterns, the study reported that direct open rates (opening the app by tapping the notification) decrease significantly as notification frequency increases. Users become less responsive to notifications when they feel overwhelmed by their frequency. Conversely, indirect open rates (opening the app within 12 hours without tapping the notification) did not show a statistically significant decline. This suggests that while users may ignore overly frequent notifications, they do not entirely lose interest in the app content.

Parallel findings in the broader literature also support this relationship between excessive notifications and reduced user receptiveness. Prior research shows that users typically tolerate up to three notifications per day before negative reactions occur, especially in learning or health-related applications — beyond that threshold, interaction rates begin to drop and perceived annoyance rises [40].

Overall, push notifications remain an essential mechanism for activating and re-engaging users; however, the evidence shows that overly frequent, non-personalised notifications can lead to a rapid decline in direct engagement and increase app abandonment. For language learning applications, notification strategies must therefore be:

- optimally timed,
- moderate in frequency,
- context-aware, and
- personalized to user behavior and progress.

Such a balanced approach helps reinforce learning habits while avoiding the user fatigue and churn associated with excessive notification frequency.

2.3.6. React Native in Mobile App

React is an open-source JavaScript library developed by Facebook, focused exclusively on building user interfaces (UIs) and forming the foundation of modern, fast, and scalable web applications [41]. The core strength of React lies in its philosophy of breaking down complex interfaces into small, independent, and reusable "components," a modular architecture that significantly speeds up development and prevents code duplication. Its most critical advantage is the Virtual DOM (Document Object Model); instead of directly manipulating the real browser structure, React efficiently updates only the changed parts of the UI, ensuring superior performance and a highly responsive user experience, which is essential for data-intensive and dynamic projects. Ultimately, adopting React for your project provides robust scalability, easier debugging due to its declarative and one-way data flow, a massive supportive community, and future-proof potential, including cross-platform development via React Native, all of which contribute to reduced technical debt and long-term project.

System Architecture and Technology Stack

The project relies on a robust backend and specialized front-end components:

Component Category	Key Technologies	Function and Rationale
Backend Framework	Spring Boot	Allows quick, efficient construction of server-side a, creating a Service-Oriented Architecture (SOA)-based REST service API .
Data Persistence	PostgreSQL	Selected for its advanced SQL compliance and superior performance capabilities in high-concurrency environments, demonstrating significantly lower latency than alternatives like MySQL.
Data Management	Hibernate + Spring Data JPA	Acts as an Object-Relational Mapping (ORM) tool, automatically generating SQL queries and eliminating the need to write raw SQL for CRUD operations.
Development Tools	Gradle, Lombok, Mockito, Swagger, Postman	Gradle manages dependencies and automates tasks; Lombok reduces boilerplate code for better readability; Mockito is used for faster, isolated unit testing by simulating dependencies; Swagger standardizes and automates interactive API documentation; and Postman is used for designing, executing, and validating RESTful API requests.
Deployment	Docker	Used for containerization to ensure a consistent runtime environment and simplify deployment, offering sufficient scalability for small to medium-sized projects compared to Kubernetes.

Content Generation	Chrome Extension & NLP	The extension extracts text/subtitles (using tools like yt-dlp and Readability.js). Natural Language Processing (NLP) components, including tokenization , lemmatization , POS tagging , and filtering using SUBTLEX word lists , are then applied to create high-quality, context-aware flashcards.
Mobile Application	React Native, Gamification	React Native is used to build the mobile application. The UI/UX is designed to be simple and engagement-oriented. Gamification elements (streaks, badges, points) are included to enhance motivation and build continuous study habits.
User Engagement	Personalization & Notifications	Systems analyse user data to create individualized recommendations. However, the research stresses that high Push Notification Frequency leads to increased uninstall rates and decreased direct open rates , requiring notifications to be moderate, personalized, and optimally timed.

Section 2: Product Vision & Scope: Syntagma

1. Product Vision

Our product vision is to build an effective and time-efficient language acquisition platform for language learners. We do this by combining passive media consumption with active vocabulary recall sessions.

While watching content or browsing the web, our extension will parse the content and give the user a comprehension score. The user can then judge the difficulty of the content and decide whether or not to engage with the content.

Easy flashcard creation based on the content the user consumes allows for later active vocabulary recall sessions.

2. Scope Boundaries

2.1 In-Scope Features

Target Language: English only

Native Language: Turkish only

Platforms: Chrome Desktop, iOS & Android (Mobile App)

AI Integration: translation, example sentence generation.

Core Workflow:

1. Parse: An NLP engine parses the website or video subtitles.
2. Providing a percentage understanding rate.
3. Capture: User can click to generate a flashcard with audio + sentence + screenshot if available.
4. Review: User can then use the mobile app to review flashcards with FSRS algorithm.

2.2 Out of scope:

1. Advanced Economy & Marketplace (Complex Gamification) While simple motivators like Streaks and XP are included, complex "virtual economies" are out of scope.

- **Excluded:** Virtual currencies (coins/gems), avatar customization, purchasing items, or an in-app store.

2. Social & Community Features The application is designed as a solitary learning tool.

- **Excluded:** Adding friends, user-to-user messaging, global leaderboards, or sharing decks with other users.

3. Real-time AI Tutoring (Chatbot) The AI integration is strictly limited to backend content processing (parsing/translating).

- **Excluded:** Conversational AI interfaces (chatting with a bot), voice-based roleplay scenarios, or grammar correction chatbots.

4. Mobile Content Creation The mobile application is a "Companion App" focused solely on the review process.

- **Excluded:** Parsing websites or creating new flashcards directly within the mobile app. All content generation happens via the Chrome Extension.

5. Multi-Language Support (MVP) The system architecture supports Internationalization, but the initial release is locked to a single pair.

- **Excluded:** Support for source languages other than English or native languages other than Turkish in the initial release.

3. Feature Breakdown

3.1 Chrome Extension

1. Content Parser

Injects itself as an interactive overlay into the DOM of the browser.

2. Message Passing

Secure passing of “flashcard” information to the backend.

3. Intuitive and non-obtrusive UI

A pop-up interface when words are hovered or clicked, displaying the dictionary definition and “Create a flashcard” button.

3.2 NLP Engine

3.2.1 Text Processing

The module is responsible for ingesting raw text from the content parser.

- Tokenization

Splits paragraphs and sentences into tokens based on the target language rules.

- Lemmatization

Reduces words to their roots to accurately match the user’s known vocabulary database.

- Stop-word filtering

Identifies and separates common grammatical structures (like “the”, “at”, “is”) to focus difficulty scoring on substantive vocabulary.

3.2.2 Language Configuration Layer

Abstracts language-specific logic to allow for future scalability (e.g., switching the Target Language from English to Spanish).

- Locale Handler: Dynamically loads the correct tokenizer and lemmatizer models based on the selected target language.

3.3 Backend

3.3.1. APIs

- Authentication & Users

Handles account creation, secure login, and user identity verification. It uses JWT.

Flashcards, collections, SRS review history, and preferences are always protected and managed by the correct authenticated user.

- Difficulty scoring

Analyzes the user’s collection of cards and returns a percentage score representing how many unique words in content the user knows.

- Cards

Manages CRUD operations related to flashcards.

- Collections of flashcards
Manages all operations related to a group of flashcards.
- Review
User reviews flashcard to perform the Spaced Repetition System.
- Statistics of usage
Users can track their statistics.
 - Accuracy
 - Progress
 - Streaks
- Synchronization to enable offline usage.
- Storage for media data manages media data to store.
 - Screenshots
 - Audio files
- AI API (Translation & Sample Sentence Generation)
It connects the desired AI to translate the word and get an example sentence with it.
- Health Check
To synchronize the necessary Space Repetition data from offline mode. It does not create a new flashcard because offline mode is only available for mobile applications, and the mobile application does not create flashcards.

3.3.2. Database

Relational SQL database that stores users, flashcards, reviews, synchronization changes, and media references. The database stores:

- User accounts and authentication information.
- Statistics for each user.
 - Streak
 - Last login
- Groups of flashcards.
- Information about flashcards.
- The information that is important for the Spaced Repetition System
 - Is the word in flashcard remembered or not
 - When was the flashcard last time reviewed
 - Repetition number
- Data that needs to be synchronized between platforms.
- References for media data for the cards.

3.3.3. Spaced Repetition System

- SRS Scheduling Algorithm (FSRS or SM-2)
- Difficulty scoring
- Date for the next review calculation

3.3.4. Logging

We have a logging feature to track application behavior, debug issues, and monitor errors.

Technologies used:

- SLF4J
- Logback

3.4 Mobile Application:

3.4.1. SRS Review Experience

It is the heart of the application, built upon the FSRS algorithm.

- Interface: A simple, button-based, and fast touch interaction structure is used, similar to applications like Duolingo.
- Content: The word, sound, screenshot of the scene if available, and an example sentence are presented on a single screen.
- Operation: The user flips the card and selects how well they know it. This feedback allows the algorithm (FSRS) to calculate the next review time.

3.4.2. Flashcard Viewer

The card detail screen provides the user with all information about the word in an integrated manner.

- Context: It displays not just the meaning of the word, but also its usage within an example sentence and its translation on the same screen.

3.4.3. Dashboard

It is the main screen that motivates the user.

- Operation: It shows the total number of words reviewed daily and allows for the review of learned words.
- Design: Clear icons and easy-to-read fonts are used to increase readability.

3.4.4. Meaning / Dictionary Integration

It is the feature that allows looking up words without leaving the application.

- When the user clicks on an unknown word in a subtitle or text, the meaning and usage examples open in a "pop-up" window.

3.4.5. Gamification

Game-like elements are used to increase motivation.

- **Streak System:** A "don't break the chain" mechanism to ensure the user logs in every day.
- **Badges:** Achievement-based rewards that are unlocked when users reach specific proficiency levels. Each badge represents a clearly defined level (e.g., A2, B1), and the system visually indicates the user's current progress within that level, such as 60% completion toward B1. This helps users understand both their current level and how close they are to the next one.

3.4.6 Push Notifications

Reminders that call the user back to the application.

- **Strategy:** Notifications are sent at the right time and are personalized for the user.
- **Frequency Setting:** Since sending notifications too frequently causes users to uninstall the app, they are sent at a non-intrusive frequency (less than 3 per day).

3.4.7. Offline Review Support

Allows studying even without the internet.

- Data is saved to the phone locally to allow studying in places like subways, planes, or where there is no internet reception.
- Progress is automatically saved when the internet becomes available.

4. User Stories:

Teacher:

As a teacher teaching English, I want to keep track of my students known words, so that I can assess their comprehension levels and assign reading material accordingly.

Self-taught learner:

As a self-taught learner, I want to have instant translations available for words I do not know, so that I can understand the plot and learn passively.

Commuter Learner:

As a learner that commutes for hours, I want to open my phone and review my flashcards, so that I can study during my downtime.

Busy Professional:

As a busy user, I want quick daily notifications that remind me to review so I don't break my learning streak.

Power User:

As a dedicated learner, I want my data to sync across devices so that I can study seamlessly on Chrome and a mobile application.

Section 3: System Architecture & Design Decisions

1. Functional Requirements

Chrome Extension

FR1.1 User Registration

The extension shall provide a registration flow that accepts email, password, and learner proficiency level and creates an account through the backend authentication API.

FR1.2 User Login

The extension shall provide a login interface that accepts email and password and authenticates the user through the backend authentication API.

FR1.3 Session Persistence

The extension shall store the authenticated session (JWT, user ID, and email) in `chrome.storage.local` so the user remains signed in across browser restarts.

FR1.4 Session Termination

The extension shall allow the user to log out and clear authentication credentials from local storage.

FR1.5 Authentication Gate

The extension shall not activate page parsing, comprehension overlays, header controls, or in-page video learning features unless the extension is enabled and a valid authentication token is present.

FR1.6 Extension Enablement

The extension shall provide a global enable/disable control that determines whether learning features are active for the user.

FR1.7 Settings Management

The extension shall persist user settings in `chrome.storage.local` and apply setting changes to active content scripts and extension pages.

FR1.8 Options Page

The extension shall provide an options page for configuring parsing, video, reader, account, word browser, flashcard, collection, and API settings.

FR1.9 Primary Workspace

The extension shall open a card-creator workspace as the primary extension UI when the user activates the toolbar action.

FR1.10 Workspace Modes

The workspace shall support home navigation, flashcard creation, flashcard editing, flashcard listing, and vocabulary browsing.

FR1.11 Standalone Auth Page

The extension shall provide a dedicated authentication page that can be opened independently of the workspace.

FR1.12 UI Localization

The extension shall support English and Turkish interface text, selected by a persisted `uiLocale` preference across extension pages.

FR1.13 Page Parsing

The extension shall traverse readable text nodes on web pages, tokenize English-like content, and enrich tokens with lexical and frequency metadata without breaking the host page layout.

FR1.14 Automatic Page Parsing

The extension shall optionally parse the active page automatically on load when the corresponding setting is enabled.

FR1.15 Syntax Highlighting

The extension shall visually mark parsed words with status-based styling for unknown, learning, known, and ignored vocabulary.

FR1.16 Inline Translations

The extension shall optionally display Turkish inline translations for parsed words on web pages when enabled.

FR1.17 Rare Word Hiding

The extension shall optionally hide rare words on parsed pages according to frequency metadata and user settings.

FR1.18 Comprehension Analysis

The extension shall calculate and display a page comprehension score and unknown-word summary based on token counts and stored lexeme status.

FR1.19 Word Interaction

The extension shall open a word popup when the user clicks a parsed word.

FR1.20 Shift Pass-Through Mode

The extension shall allow the user to hold Shift to interact with the underlying page through parsed overlays.

FR1.21 Keyboard Shortcuts

The extension shall provide keyboard shortcuts for common learning actions on parsed pages and in video mode.

FR1.22 SPA Navigation Handling

The extension shall reinitialize or update learning features when single-page applications change URL state without a full reload, including supported YouTube and Netflix watch pages.

FR1.23 Word Status Updates

The extension shall allow the user to set a word's status to unknown, learning, known, or ignored from the word popup and related UI surfaces.

FR1.24 Word Knowledge Synchronization

The extension shall synchronize lexeme create, update, delete, and bulk status changes with the backend word-knowledge API for authenticated users.

FR1.25 Known Words Browser

The extension shall provide a searchable and filterable interface for viewing and managing the user's vocabulary by status.

FR1.26 CEFR Learner Levels

The extension shall allow the user to select a learner proficiency level mapped to CEFR bands, including A1, A2, B1, B2, C1, and C2.

FR1.27 CEFR Level Intake

When the learner level changes, the extension shall request backend CEFR intake for that level and update local known-word state accordingly.

FR1.28 Local Dictionary Lookup

The extension shall provide fast local dictionary translation lookup from an IndexedDB store populated from a bundled dictionary.json resource.

FR1.29 External Dictionary Links

The extension shall allow the user to open selected words in external dictionary providers, including Tureng, Cambridge, Oxford, and Merriam-Webster.

FR1.30 Word Audio Playback

The extension shall retrieve pronunciation audio for a word from the Dictionary API when available and fall back to browser speech synthesis when not.

FR1.31 Word Popup Learning Actions

The word popup shall support status changes, dictionary display, frequency display, audio playback, external dictionary links, and flashcard actions.

FR1.32 AI Word Explanation

The extension shall request streamed AI word explanations in sentence context through the backend for authenticated users.

FR1.33 AI Sentence Explanation

The extension shall request streamed AI grammar or usage explanations for a sentence through the backend for authenticated users.

FR1.34 AI Sentence Translation

The extension shall request streamed AI Turkish translation for a sentence through the backend for authenticated users.

FR1.35 AI Example Sentence Generation

The extension shall request an AI-generated example sentence for a target word through the backend and allow the user to accept or edit it during flashcard creation.

FR1.36 Backend-Only AI Access

The extension shall route all AI features through authenticated backend endpoints and shall not call third-party AI APIs directly in production use.

FR1.37 Flashcard Creation

The extension shall allow users to create contextual flashcards from web pages, video subtitles, the eBook reader, and the workspace.

FR1.38 Flashcard Editing

The extension shall allow users to edit existing flashcards, including text fields, collection assignment, and optional screenshot or audio media operations.

FR1.39 Flashcard Listing

The extension shall display the user's flashcards and support selection for review, editing, and export actions.

FR1.40 Flashcard Synchronization

The extension shall create, fetch, update, and delete flashcards through the backend API for authenticated users.

FR1.41 Flashcard Payload Content

A flashcard created or edited by the extension shall be able to include lemma, surface form, source sentence, optional example sentence, Turkish meaning, source URL, source title, knowledge status, tags, deck/collection association, screenshot data, sentence audio data, and video timestamp when available.

FR1.42 Collection Management

The extension shall allow authenticated users to create, list, select, and delete flashcard collections through the backend API.

FR1.43 Active Collection Selection

The extension shall allow the user to choose an active collection/deck used for new flashcard creation.

FR1.44 Context Menu Actions

The extension shall provide context menu entries for opening the workspace and initiating learning actions from the current page context.

FR1.45 YouTube Subtitle Capture

The extension shall extract subtitle cues from YouTube watch pages using timed-text retrieval, player-response parsing, and text-track fallback.

FR1.46 Netflix Subtitle Capture

The extension shall extract subtitle cues from Netflix watch pages using a page-context network interceptor and TTML/manifest parsing with DOM fallback recovery.

FR1.47 Dual Subtitles

The extension shall support displaying target-language and support-language subtitle tracks simultaneously when dual-subtitle mode is enabled.

FR1.48 Auto-Pause Modes

The extension shall support configurable auto-pause behavior during video learning, including off, before cue, after cue, before-and-after cue, and rewind-and-pause modes.

FR1.49 Subtitle Navigation

The extension shall allow the user to navigate among subtitle cues using keyboard shortcuts and UI controls.

FR1.50 Subtitle Import

The extension shall allow the user to import external subtitle files in SRT or VTT format for supported video learning surfaces.

FR1.51 Subtitle Sidebar

The extension shall optionally display a subtitle sidebar with clickable cues and word interaction support during video learning.

FR1.52 Sentence Audio Capture

The extension shall record a short audio clip aligned to subtitle cue timing from the active video element and attach it to flashcard data when supported by the media source.

FR1.53 Standalone Video Player

The extension shall provide a dedicated video-player page that supports local video files, imported subtitles, playback controls, and the same subtitle interaction features as in-page video mode.

FR1.54 Media Upload and Retrieval

The extension shall upload flashcard media to backend presigned storage when required and retrieve signed media URLs for display or playback.

FR1.55 eBook Reader

The extension shall provide a reader page for opening and reading EPUB books within the extension.

FR1.56 eBook Library Synchronization

The extension shall upload EPUB files to backend storage through presigned URLs, register book metadata, and download books for reading through authenticated backend URLs.

FR1.57 Reader Vocabulary Interaction

The eBook reader shall parse chapter text with the same vocabulary highlighting, popup, status updates, and flashcard creation flows used on web pages.

FR1.58 Reader Settings

The eBook reader shall support configurable font size, inline translations.

FR1.59 Reading Progress Persistence

The extension shall persist per-book reading position and progress locally for resumed reading sessions.

FR1.60 Whole-Book Analysis

The extension shall support vocabulary analysis across an EPUB book or chapter for comprehension reporting in the reader workspace.

FR1.61 Message-Based Architecture

The extension shall communicate among content scripts, the service worker, and extension pages through typed runtime messages with asynchronous command-response handling.

FR1.62 Background Orchestration

The extension background service worker shall handle settings persistence, lexeme and flashcard storage, dictionary indexing, AI orchestration, backend API calls, media capture coordination, and extension page launching.

FR1.63 Event Broadcasting

The extension shall broadcast settings updates, vocabulary status changes, and AI stream events to listening UI contexts.

FR1.64 Manifest V3 Compliance

The extension shall operate as a Chrome Manifest V3 extension using a service worker background script and isolated content-script execution.

FR1.65 Host Permissions

The extension shall request permissions required for storage, scripting, tabs, context menus, window management, tab capture, and access to configured backend and dictionary endpoints.

NLP

FR2.1 Raw Text Ingestion

The NLP pipeline shall ingest raw text from web page DOM content, video subtitles, and EPUB chapter text supplied by the extension.

FR2.2 Sentence and Token Segmentation

The NLP pipeline shall segment text into sentences and tokens using language-appropriate rules for English web and subtitle content.

FR2.3 Lemmatization

The NLP pipeline shall reduce inflected English word forms to lemmas for vocabulary matching and flashcard identity.

FR2.4 Irregular Form Normalization

The NLP pipeline shall normalize common irregular English forms using a maintained irregular-form map and caching.

FR2.5 Contraction Expansion

The NLP pipeline shall expand English contractions into constituent lemmas and resolve composite learning status from their parts.

FR2.6 Phrase Recognition

The NLP pipeline shall recognize configured multi-word phrases and treat them as phrase-level lexical units.

FR2.7 Frequency Enrichment

The NLP pipeline shall assign frequency rank and frequency band to tokens using a bundled English frequency dataset.

FR2.8 CEFR Classification Support

The NLP pipeline shall associate vocabulary with CEFR-oriented word lists used for learner-level intake and difficulty estimation.

FR2.9 Vocabulary Status Resolution

The NLP pipeline shall map tokens and phrases to user lexeme status for unknown, learning, known, and ignored states.

FR2.10 Comprehension Scoring

The NLP pipeline shall compute comprehension percentages and unknown-token statistics from token counts and resolved vocabulary status.

FR2.11 Rare Word Filtering

The NLP pipeline shall support filtering or hiding rare tokens based on frequency metadata and user settings.

FR2.12 Flashcard Data Preparation

The NLP pipeline shall produce structured flashcard input containing lemma, surface form, source sentence, source context metadata, and optional media references.

FR2.13 Non-Destructive DOM Output

The NLP pipeline shall generate overlay markup and classes for parsed content without replacing the underlying page structure.

FR2.14 English–Turkish Language Pair

The current NLP configuration shall target English content with Turkish support-language output for translations and learner assistance.

FR2.15 Synchronous Content-Script Operation

The NLP components used in content scripts shall operate synchronously within the content-script bundle constraints required by the extension architecture.

Backend

FR-3.1: User Authentication & Authorization

- FR-3.1.1: The system must allow users to create accounts.
- FR-3.1.2: The system must allow users to securely log in with their email address and password.
- FR-3.1.3: The system should generate a JWT (JSON Web Token) for the user after a successful login.
- FR-3.1.4: The system must perform JWT validation on all protected API requests.
- FR-3.1.5: The system should process each request only under the name of the corresponding authenticated user.
- FR-3.1.6: The system must store the user's basic profile information (user ID, email, preferences).

FR-3.2 User Information:

- FR-3.2.1: The system should be able to save the user's learning preferences (notification settings, target daily word count Daily new card limit, etc.).
- FR-3.2.2: The system needs to update the user's last login time and active streak information.

FR-3.3: Flashcard Management

- FR-3.3.1: The system must support creating new flashcards from data received via the Chrome Extension.
- FR-3.3.2: Each flashcard should contain the following information:
 - Word (lemma)
 - Source of word's sentence
 - Example sentence (OPTIONAL)
 - Translation
- FR-3.3.3: The system must be able to perform CRUD operations (create, read, update, delete) on the user's flashcards.
- FR-3.3.4: The system must ensure that flashcards are accessible only by the authorized user.

FR-3.4: Flashcard Collections

- FR-3.4.1: The system should allow users to group flashcards under collections.
- FR-3.4.2: The system must support creating, deleting, and updating collections.
- FR-3.4.3: The system must support adding or removing a flashcard from a collection.

FR-3.5: AI Integration (Translation & Example Generation)

- FR-3.5.1: The system should be able to generate AI-based translations for the selected sentence.
- FR-3.5.2: The system should generate an example sentence containing the word.
- FR-3.5.3: The system should generate sentence explanation for selected sentence.
- FR-3.5.4: The system should generate word explanation for selected sentence's context.
- FR-3.5.5: AI services should only be called from the backend (the frontend cannot access them directly).

FR-3.6: Spaced Repetition System (SRS)

- FR-3.6.1: The system must store the FSRS (Free Spaced Repetition Scheduler) parameters (stability, difficulty, retrievability) in a dedicated SRS state for each flashcard.
- FR-3.6.2: The system should run the FSRS algorithm based on the user's review rating (e.g., GOOD, AGAIN).
- FR-3.6.3: The system must calculate the next review date for the flashcard after each review.
- FR-3.6.4: The system should be able to generate a daily review list for the user.

FR-3.7: Review History Tracking

- FR-3.7.1: The system must record each review process along with the date, result, and the option used.
- FR-3.7.2: The system should be able to use historical review data to generate statistics.

FR-3.8: Statistics & Progress Tracking

- FR-3.8.1: The system should calculate the following user statistics:

- Daily review count
- Accuracy rate
- Active streak duration
- FR-3.8.2: The system should provide these statistics via API.

FR-3.9: Synchronization & Offline Support

- **FR-3.9.1:** The backend system must provide endpoints to push offline events and pull the latest server state to support the mobile application's offline-first architecture.
- **FR-3.9.2:** Conflict resolution based on timestamps must be handled by the mobile client before pushing the final synchronized state to the backend.

FR-3.10: Media Storage Management

- FR-3.10.1: The system must store media references for audio and screenshot files on flashcards.
- FR-3.10.2: The system must ensure that media files can be accessed securely from the backend.

FR-3.11: Health Check & Monitoring

- FR-3.11.1: The system should provide a health check endpoint to verify that the backend services are up and running.
- FR-3.11.2: This endpoint should be used for synchronization and the mobile application.

FR-3.12: Logging & Error Tracking

- FR-3.12.1: The system must log all critical backend operations.
- FR-3.12.2: Errors should be logged at appropriate log levels (INFO, WARN, ERROR).
- FR-3.12.3: Logging should be done in a way that does not disclose user data.

FR-3.13: Word Knowledge Management

- FR-3.13.1: The system must track the knowledge status (e.g., KNOWN, LEARNING) of individual words (lemmas) for each user across the entire platform.
- FR-3.13.2: The Spaced Repetition System should automatically update a word's status to KNOWN after a certain number of successful reviews (e.g., reaching a specific interval in FSRS).
- FR-3.13.3: The system must allow users to manually mark specific words or a batch of words as already known.
- FR-3.13.4: The system should support marking a set of words as known based on the user's current proficiency level (e.g., automatically marking A1 level words as KNOWN)

Mobile Application

FR-4.1: User Authentication:

The mobile application shall allow users to log in using their existing account credentials. Authenticated users shall only be able to access their own flashcards, review history, and statistical data.

FR-4.2: Flashcard Synchronization

The mobile application shall synchronize flashcards, review schedules, and user progress data with the backend system.

This synchronization shall ensure data consistency between the Chrome extension and the mobile application.

FR-4.3: SRS-Based Review Sessions

The mobile application shall provide vocabulary review sessions based on a Spaced Repetition System (SRS – FSRS algorithm).

Flashcards presented during review sessions shall be selected according to the review schedule determined by the system.

FR-4.4: Flashcard Detail View

The mobile application shall provide a detailed view for each flashcard.

This view shall display the word's meaning, audio pronunciation (if available), example sentence, and usage context on a single screen.

FR-4.5: User Self-Assessment

The mobile application shall allow users to indicate whether they remember a flashcard or not.

The feedback provided by the user shall be used to update the review schedule of the SRS algorithm.

FR-4.6: Offline Operation Support (Offline Mode)

The mobile application shall be able to operate in environments without an internet connection (e.g., subway, airplane).

For this purpose, required data shall be stored locally on the device and automatically synchronized when an internet connection becomes available.

FR-4.7: User Progress Dashboard

The mobile application shall provide a progress dashboard that displays the user's daily reviewed flashcard count, learning streak information, and overall learning and progress statistics.

FR-4.8: Streak System

The mobile application shall provide a streak system that tracks usage and review activities performed on consecutive days in order to encourage daily application usage.

FR-4.9: Badge and Achievement System

The mobile application shall unlock badges when users reach specific language proficiency levels (e.g., A2, B1).

The user's progress within the current proficiency level shall be visually represented.

FR-4.10: Notification System

The mobile application shall send push notifications to remind users of scheduled review times.

Notification frequency shall be configurable by the user and limited to a non-intrusive level.

FR-4.11: Local Data Storage

The mobile application shall store flashcards, review history, and synchronization-related data locally on the device to support offline usage.

FR-4.12: Synchronization and Error Handling

The mobile application shall be able to detect errors that occur during synchronization.

When the connection is restored, failed synchronization operations shall be retried automatically.

User progress shall not be lost due to temporary connectivity issues.

FR-4.13: Flashcard Knowledge Status Tracking

The mobile application shall allow users to assign a knowledge status to each flashcard as "known," "learning," "unknown," or "ignored."

The selected knowledge status shall be stored in the backend system and synchronized across all platforms.

2. Use Case Diagrams: Chrome Extension:

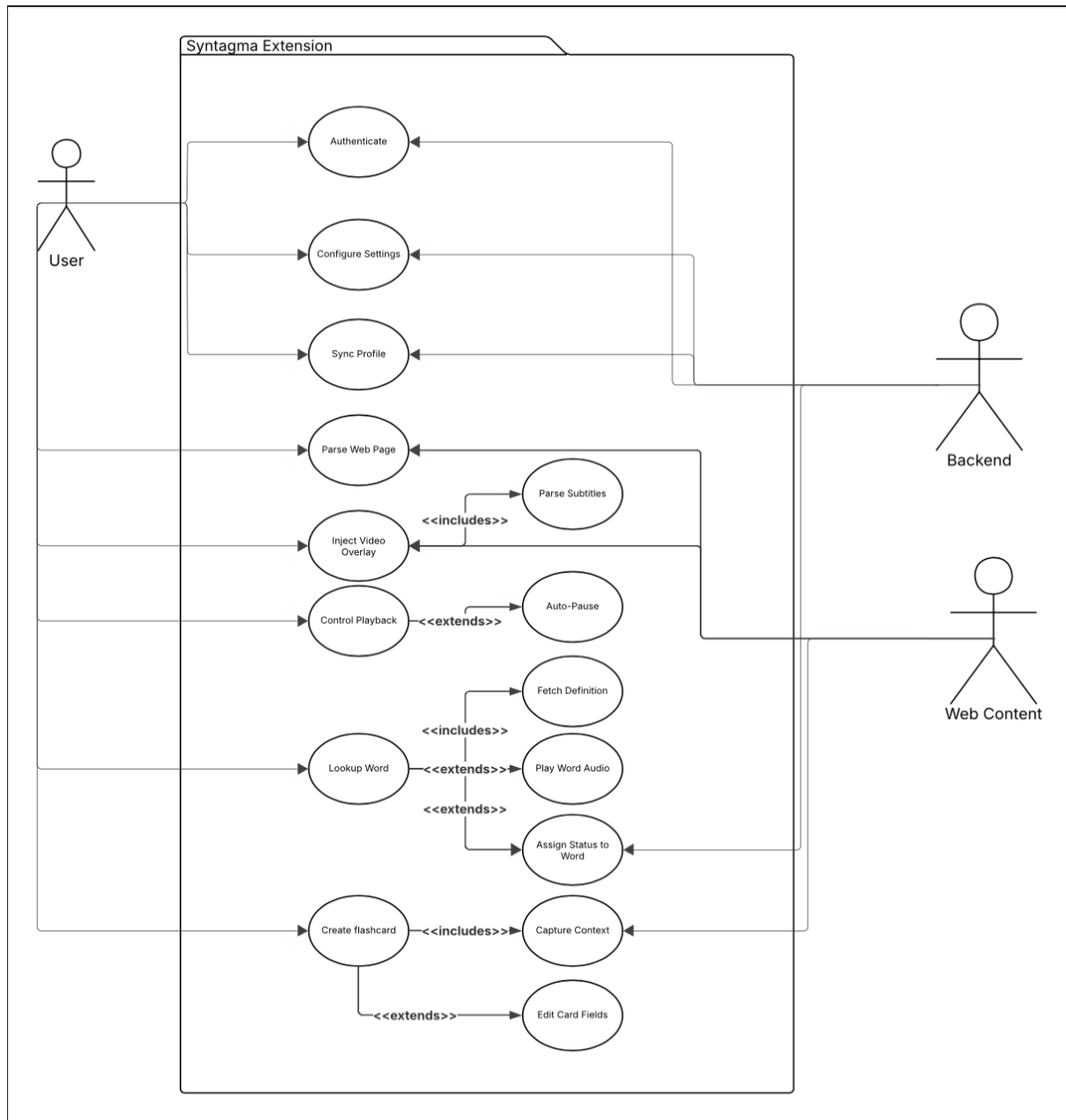


Figure 3 Use Case Diagram of Chrome Extension

Mobile Application:

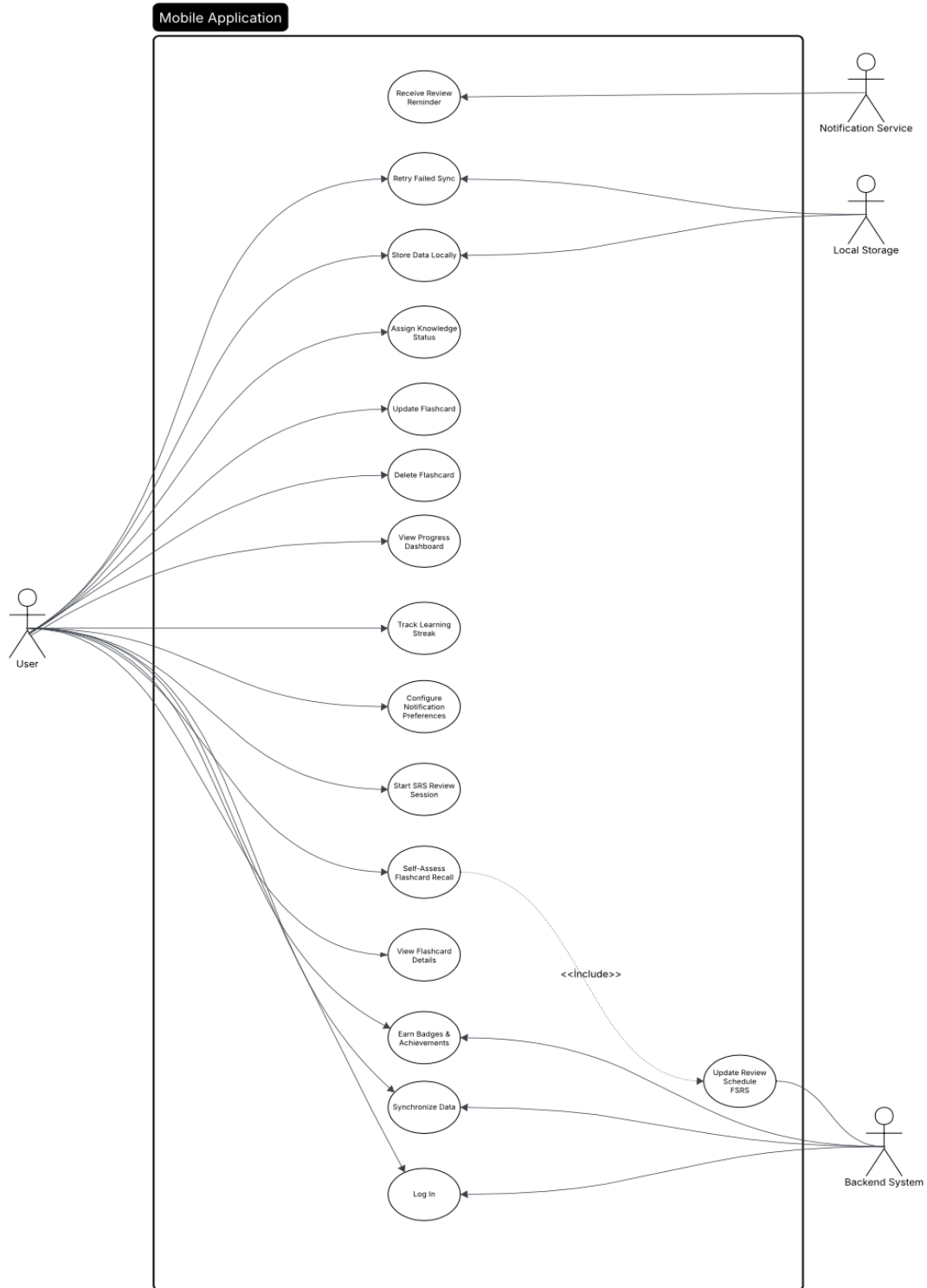


Figure 4 Use Case Diagram of Mobile Application

3. Data Flow Diagrams:

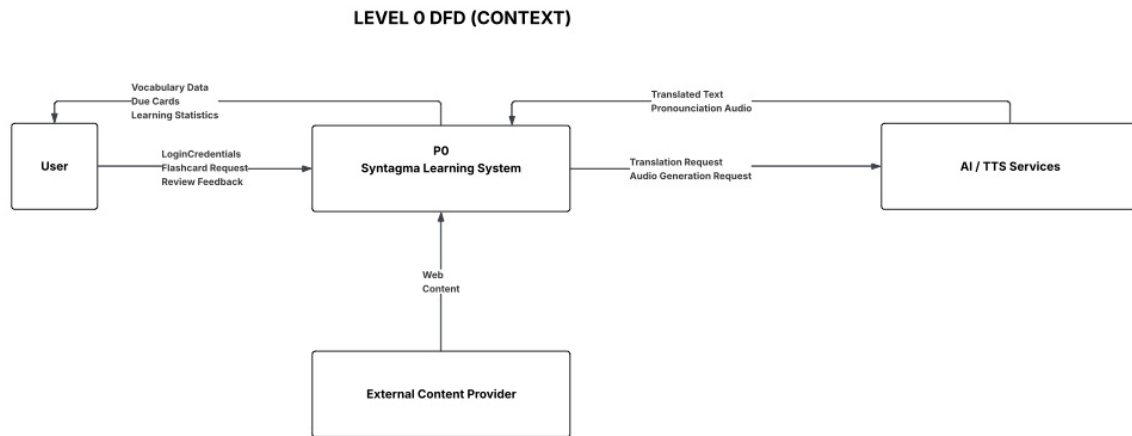


Figure 5 Data Flow Diagram of Syntagma Learning System Level 0

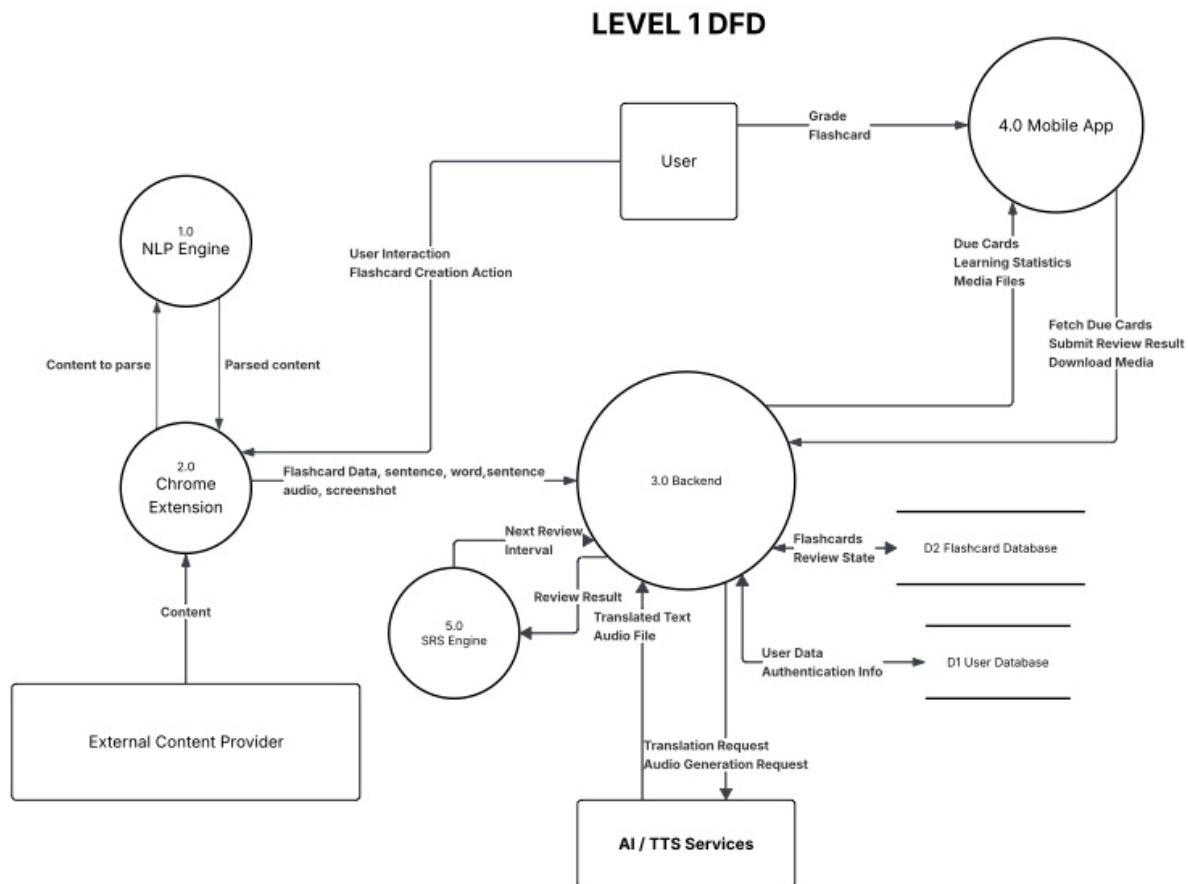


Figure 6 Data Flow Diagram of Syntagma Learning System Level 1

Section 4: Implementation Details

1. Introduction

Syntagma is an AI-powered, multi-platform language learning application designed to optimize vocabulary acquisition through advanced Spaced Repetition System (SRS) principles. The primary goal of the project is to lower cognitive load and facilitate long-term memory retention for Turkish-speaking users learning English. By integrating learning directly into the user's daily digital consumption—such as reading web articles or watching videos on platforms like YouTube—Syntagma provides a seamless, context-rich educational experience.

The Syntagma ecosystem is composed of three primary pillars:

- **Backend Application:** The robust core built on Spring Boot and PostgreSQL, responsible for managing user data, securing authentication via JWT, and processing the sophisticated FSRS (Free Spaced Repetition Scheduler) learning algorithm.
- **Browser Extension:** A Chrome Manifest V3 client that acts as the primary data capture tool. It features real-time text tokenization, contextual word popups, and advanced video subtitle interception to generate flashcards organically while the user browses the web.
- **Mobile Application:** A cross-platform app developed within the React Native/Expo ecosystem, allowing users to review their synced flashcards on the go, utilizing visual narratives and audio support to reinforce memory.

This document details the architectural decisions, technology stacks, core data models, and specific implementation workflows for each of these three subsystems, outlining how they function individually to support the overarching educational platform.

2. Backend Application

This section provides an overview of the backend system structure and technologies used in the project.

2.1 Technology Stack

The backend system is implemented using modern and scalable technologies:

- **Framework:** Spring Boot [42]
- **Database:** PostgreSQL [43]
- **Security:** Spring JWT [44]
- **API Documentation:** Swagger [45] / Postman [46]
- **Build Tool:** Gradle [47]
- **Hibernate** [48]

2.2 Architectural Structure

The backend follows a standard multi-layered architecture:

- **Controllers (/controller):** Exposes REST API endpoints. Controllers use DTOs for requests and responses.
- **Services (/service):** Encapsulates all business logic, algorithms, and orchestration.
- **Repositories (/repository):** Spring Data JPA interfaces interacting directly with the PostgreSQL database.
- **Entities (/entity):** JPA mapped domain classes defining the database schemas.
- **DTOs (/dto):** Strict request/response models to prevent entity leakage and simplify communication payloads.
- **Security (/security):** Stateless JWT filters enforcing authentication contexts on secured endpoints.
- **Configuration (/config):** Centralized configuration for security, OpenAPI, AI clients, and storage.

2.3 Core Business Modules

2.3.1 Advanced Spaced Repetition (FSRS Algorithm)

System uses Java implementation of the FSRS Algorithm, the modern standard recently adopted by Anki [49].

- **FsrsAlgorithm.java:** Implements standard FSRS math, computing Difficulty, Stability, and Retrievability. 90% threshold rate. [50]
- **SrsState & ReviewLog:** Tracking individual learning states per flashcard.

2.3.2 Flashcards & Collections

- Hierarchical data structures using **Collection**, **CollectionItem**, and **Flashcard**.
- Includes **MediaAsset** references to bind images and audio logically to learning content without bloating the central database schemas.

2.3.3 Sync and Offline Support

- **SyncEvent**-based push/pull sync for client devices.
- Note: Entity-level sync processing is currently a placeholder for future expansion.

2.3.4 AI-Assisted Learning

- AI services for word explanations, sentence explanations, and translations.
- Requests are routed through a configurable external AI API provider.

2.3.5 Media and Ebook Management

- **MediaAsset** and **StorageService** provide S3-compatible presigned upload and download URLs.

- Ebook ingestion and progress tracking support reading-based learning workflows.

2.3.6 Word Knowledge Tracking

- WordKnowledge stores per-user vocabulary knowledge and related stats.

2.4 Security & Exception Handling

- JWT: Auth endpoints distribute JWT tokens; JwtAuthenticationFilter validates and bridges the user identity to secured resources.
- GlobalExceptionHandler: Provides uniform JSON-formatted error responses for EntityNotFound, validation failures, duplicates, and generic uncaught exceptions.

2.5 Configuration Highlights

- application.yml defines database, JWT, storage, logging, and AI settings.
- OpenAPI configuration documents secured endpoints with bearer authentication.
- SrsState & ReviewLog: Tracking individual learning states per flashcard.

3. Chrome Extension

3.1 Purpose and Scope

This section defines implementation details of the Syntagma browser extension under the extension/ directory.

Covered areas:

- Runtime architecture and component boundaries
- Build and packaging pipeline
- Core data and storage contracts
- Messaging and event-driven communication
- Content parsing and overlay logic
- Subtitle capture and video interaction subsystem
- Flashcard, dictionary, AI, and backend integration points

Test status and operational constraints

3.2 System Context

The extension is a Chrome Manifest V3 client that supports English learning for Turkish-speaking users by:

- tokenizing and lemmatizing page text
- applying word-level knowledge overlays
- showing contextual word popups
- capturing and processing subtitles

- creating flashcards and syncing learning state

Execution contexts:

1. Background Service Worker
2. Content Script Runtime
3. Extension Pages (Card Creator / Options / Auth / Reader / Video Player)

3.3 Technology Stack

- **Language:** TypeScript [51]
- **UI:** React 19, React DOM 19 [52]
- **Build:** Vite 8, Rollup outputs [53]
- **Extension Standard:** Chrome Manifest V3 [54]
- **Testing:** Vitest [55]
- **NLP support:** compromise [56]
- eBook reader: epubjs
- **Utilities:** uuid [57], lz-string [58], react-markdown [59]

3.4 Runtime Architecture

3.4.1 Background Layer

Responsibilities:

- process runtime message commands
- persist settings and lexical state
- orchestrate AI calls and stream chunks to tabs
- lookup translations from IndexedDB dictionary
- sync word status and flashcards to backend API
- create and handle extension context menu
- open options, card-creator, auth, reader, and video-player windows
- tab screenshot capture and tabCapture stream IDs for media

Notable implementation behaviour:

- dictionary population starts on worker startup
- keep-alive interval using `chrome.runtime.getPlatformInfo`
- message sender ID verification against own extension ID
- setting/status broadcasts to all tabs
- JWT refresh via X-Refreshed-Token response header
- cached signed URLs for backend flashcard media assets

3.4.2 Shared Layer

Responsibilities:

- central type definitions
- typed message envelopes
- storage helpers with defaults and user-scoped keys
- frequency table initialization and lookups
- lemmatization and irregular normalization cache
- backend AI client (backend-ai.ts)
- CEFR word lists and intake helpers
- i18n dictionary (en/tr)

3.4.3 Content Layer

Responsibilities:

- parse lifecycle orchestration
- token extraction and overlay rendering
- toolbar UI and page analysis
- popup-based word actions
- video subtitle overlay and transcript interactions
- Authentication gate: On initialization, the content script returns immediately if enabled is false or authToken is absent. Parsing, header bar, overlays, and video mode require a signed-in user.
- Video activation: Video mode initializes on YouTube watch and Netflix watch URLs after settings and lexemes load.

3.4.4 Extension Pages Layer

Extension pages are React applications opened as Chrome extension windows or tabs:

- card-creator.html — primary workspace (toolbar icon opens this; not default_popup)
- options.html — settings (manifest options_ui)
- auth.html — login and registration
- reader.html — EPUB eBook reader
- video-player.html — standalone video learning surface
- popup.html — built but not registered as toolbar popup in manifest.json

3.5 Core Data Model

3.5.1 User settings

Stored at chrome.storage.local.userSettings (merged with DEFAULT_SETTINGS).

Includes:

- Global: enabled, learnerLevel, supportLocale, targetLanguage, uiLocale (en | tr)
- Page parsing: showComprehensionHeader, showInlineTranslations, showLearningStatusColors, hideRareWords, autoParseOnLoad

- Reader: readerEnableInlineTranslations, readerShowLearningStatusColors, readerAutoParseChapterOnOpen, readerDefaultFontSize, readerLineHeight, readerTheme
- Video: autoPauseMode, subtitleDualMode, obscure modes, subtitle sizes, overlay opacity, pause/resume, scene skip, reveal options, offsets, sidebar
- AI / integrations: aiModel, aiApiKey (reserved), apiBaseUrl, authToken, authEmail, authUserId, activeCollectionId, activeCollectionName

3.5.2 Lexeme state

Stored under user-scoped key lexemes_{authUserId} when authUserId is set, otherwise lexemes.

Schema key: normalized lemma (string)

Schema value: LexemeEntry — status (unknown | learning | known | ignored), surface, type (word | phrase), trMeaning, audioUrl, cefr, frequency metadata, timestamps

Updates persisted locally and synced to /api/word-knowledge when authenticated.

3.5.3 Flashcards

Stored under flashcards_{authUserId} (or flashcards) as FlashcardPayload[].

Fields: id, lemma, surfaceForm, sentence, exampleSentence, sourceUrl, sourceTitle, trMeaning, knowledgeStatus, audioUrl, screenshotDataUrl, sentenceAudioDataUrl, videoTimestamp, deckName, tags, collectionId(s)

Created from word popup, content quick-add, video overlay, reader, and card-creator; synced with backend collections and media.

3.5.4 Dictionary store

IndexedDB: SyntagmaDictionaryDB, object store translations

Populated at service-worker startup from dictionary.json (web-accessible resource)

Lookup via LOOKUP_DICTIONARY message → dictionary-db.ts

dictionary.json must be placed at extension/dictionary.json before build (not committed to repo)

3.5.5 Auxiliary storage keys

flashcardMediaUrls — cached signed backend media URLs

flashcardScreenshotUrls — legacy screenshot cache

cardCreatorDraft:* — edit-session draft cards

comprehensionStats — per-page comprehension snapshots

3.6 Message Contract and Communication

Message model is command-response with async handling.

Extension command groups include:

- settings get/set
- single and bulk word status updates; upsert/delete word knowledge
- dictionary and audio lookup; word frequency lookup
- AI requests (explain word, explain sentence, translate, generate example)
- flashcard create/update/fetch/delete; collections fetch/create
- options/card creator/auth/reader/video-player window actions
- LOGIN, REGISTER, LOGOUT
- CAPTURE_TAB_SCREENSHOT, GET_TAB_CAPTURE_STREAM_ID
- OPEN_EXTERNAL_DICTIONARY (Tureng, Cambridge, Oxford, Merriam-Webster)

Background event groups include:

- AI stream chunk/done/error
- status changed notifications
- settings updated notifications

3.7 Page Parsing and Overlay Pipeline

3.7.1 Parsing flow

- TreeWalker traversal over document text nodes
- skipped tag and hidden element filtering
- regex tokenization for English-like tokens
- lemmatization and frequency enrichment
- contraction expansion and composite status resolution
- status mapping from lexeme state
- chunked processing with requestIdleCallback fallback

3.7.2 Overlay injection

- sentence wrappers: span.syn-sentence
- word wrappers: span[data-syn="lemma"]
- status class assignment (unknown/learning/known/ignored)
- optional inline TR translation tags
- scoped style injection and sentence-hover highlight behavior

3.7.3 User interaction logic

- click word to open popup
- shift-mode to pass clicks through overlays
- keyboard shortcuts (Alt+X)
- SPA navigation handling for YouTube and Netflix

3.8 Word Popup and Learning Actions

Word popup supports:

- status updates

- dictionary translation display
- frequency badge
- word audio playback (Dictionary API URL or speech synthesis fallback)
- external dictionary links

AI commands:

- explain word in sentence context
- explain sentence grammar
- translate sentence
- quick flashcard creation
- Quick flashcard creation may include screenshotDataUrl and sentenceAudioDataUrl from video context; card creator opens via OPEN_CARD_CREATOR with URL parameters or a draft storage key in edit mode.

Streaming behaviour:

- background dispatches chunked responses by requestId
- popup aggregates until done/error event

3.9 Video Subsystem

3.9.1 Mounting strategy

Video overlay mounts on document.body using fixed-position host to avoid player-overlay pointer-event conflicts.

3.9.2 Subtitle capture sources

YouTube sequence:

- timedtext API formats
- InnerTube/player-response extraction
- textTracks fallback

Netflix sequence:

- page-context interception of fetch/XHR
- manifest parsing for subtitle chunk URLs
- TTML chunk aggregation
- unified cue dispatch + DOM cache recovery path

3.9.3 Synchronization model

- YouTube auto-offset detection for subtitle alignment
- native subtitle layers hidden when extension subtitles are active

3.9.4 Interaction capabilities

- clickable subtitle words with popup integration
- multiple auto-pause modes

- silent-gap scene-skip strategies
- subtitle import (SRT/VTT)
- Netflix-safe seek via injected internal API

3.9.5 Built-in Video Player Page

- File: video-player.html → VideoPlayerApp.tsx. Opened via OPEN_VIDEO_PLAYER.
- Dedicated HTML5 video surface independent of YouTube/Netflix DOM
- Import SRT/VTT subtitles; tokenized clickable words
- Shared auto-pause, reveal, and popup flows with in-page video mode
- Flashcard creation with video-player source metadata

3.9.6 Audio Capture for Flashcards

In-page video (audio-recorder.ts):

- Uses HTMLVideoElement.captureStream() and MediaRecorder (no tabCapture for cue clips)
- Records audio aligned to subtitle cue times (max 30 s per cue)
- Produces WebM data URLs as sentenceAudioDataUrl on flashcards

Background service worker:

- tabCapture permission for GET_TAB_CAPTURE_STREAM_ID
- tabs.captureVisibleTab for CAPTURE_TAB_SCREENSHOT
- Backend MediaAsset signed URLs cached in flashcardMediaUrls

3.10 Integrations

3.10.1 Dictionary audio API

Default pronunciation source: <https://api.dictionaryapi.dev/api/v2/entries/en/{word}>

3.10.2 Backend sync

Default base fallback: <https://syntagma.omerhanyigit.online>

Used for:

- word knowledge sync (single, batch, and level intake)
- flashcard create/list/update/delete and collections
- AI explain/translate/generate-example endpoints
- media asset URLs for flashcard screenshot and audio
- Token refresh: X-Refreshed-Token header updates authToken in userSettings automatically.

3.10.3 AI Example Sentence Generation

Message: GENERATE_EXAMPLE_SENTENCE

Endpoint: POST /api/ai/generate-example (requires authToken)

Used from card creator to populate exampleSentence on FlashcardPayload

Single request-response (not streamed)

3.11 Storage and Persistence Summary

chrome.storage.local:

- userSettings — global UserSettings
- lexemes or lexemes_{userId} — Record<lemma, LexemeEntry>
- flashcards or flashcards_{userId} — FlashcardPayload[]
- comprehensionStats — page comprehension snapshots
- flashcardMediaUrls — cached backend media URLs
- cardCreatorDraft:* — ephemeral edit drafts
- User scoping via userScopedKey(base, authUserId) isolates data per account on the same browser profile.
- IndexedDB: SyntagmaDictionaryDB/translations — populated from dictionary.json.

3.12 Testing and Quality Status

See Section 3.18 for the current automated test inventory and manual QA prerequisites.

3.13 Operational Notes and Known Limitations

- dictionary.json is not in the repository; add extension/dictionary.json before npm run build.
- Content-script features require authToken (sign in via auth.html or card creator).
- Toolbar icon opens card-creator.html, not popup.html.
- Netflix capture depends on document_start interceptor; site changes may require updates.
- Production AI uses backend endpoints; client OpenRouter [60](ai.ts) is unused.
- forvoApiKey exists in settings only; pronunciation uses Dictionary API and speech synthesis.
- index.html / App.tsx is a dev scaffold, not a shipped extension entry.

3.14 Extension Pages

3.14.1 Workspace (card-creator.html)

CardCreatorApp.tsx — central hub: home panel, flashcard create/edit, word browser, flashcard list, inline auth, collection picker, AI example sentences. Opened by toolbar click, context menu, and OPEN_CARD_CREATOR.

3.14.2 Auth Page (auth.html)

AuthApp.tsx — login/register, password rules, learner level on registration, persists JWT via LOGIN/REGISTER messages. Can run standalone or inline in card creator.

3.14.3 Settings Page (options.html)

OptionsApp.tsx — manifest options_ui: parsing, video, reader, AI URL, account, word browser, flashcard management.

3.14.4 eBook Reader Page (reader.html)

ReaderApp.tsx — epubjs EPUB reader, chapter overlays, whole-book analysis (reader-analysis.ts), comprehension stats, word popup and flashcard actions, light/sepia/dark themes.

3.14.5 Popup Page (popup.html)

PopupPageApp.tsx — compact enable toggle; built but not bound as default_popup in manifest.

3.15 Internationalization (i18n) System

Module: src/shared/i18n.tsx

Locales: English (en), Turkish (tr); persisted as UserSettings.uiLocale

t(key, locale) and useT hook; LocaleToggle component

Used in card creator, options, auth, reader, and feature description strings

3.16 Contraction and Morphological Handling

Contractions (parser.ts, video/tokenizer.ts):

- CONTRACTIONS map surfaces like don't to constituent lemmas [do, not]
- resolveContractionStatus: unknown if any part unknown; else learning if any learning; else known
- Lemmatization (lemmatizer.ts):
- compromise for verb infinitive and noun singular forms
- in-memory cache; large IRREGULAR_MAP pre-seeded at module load
- synchronous API required for content-script IIFE bundle
- Multi-word phrases matched from assets/phrases-en.json as type phrase.

3.17 CEFR Level Support

LearnerLevel maps to CEFR: beginner→A1, elementary→A2, intermediate→B1, upper-intermediate→B2, pre-advanced→C1, advanced→C2

Word lists: src/assets/levels/a1.txt ... c2.txt and cefr-words-en.json

cefr-intake.ts: POST /api/word-knowledge/level on level change; syncs local lexemes from backend KNOWN set

pre-advanced selects C1 corpus for intake between B2 and C2.

4. Mobile Application

4.1 Purpose and Scope

Syntagma has a mobile application developed within the Expo ecosystem that optimizes the language learning process based on Spaced Repetition System (SRS) principles. The application aims to facilitate the transfer of vocabulary into long-term memory through its academic-focused design and machine-learning-supported algorithms. Focusing initially on the English-Turkish language pair, the project offers a sustainable study discipline by reducing cognitive load through visual narratives and gamification elements.

4.2 Architecture

The application is built on React 19 and React Native [61] 0.81 technologies within the Expo [62] ecosystem, offering cross-platform support (iOS/Android).

- **Modular Structure:** Navigation management is structured in a hierarchical model (Authentication Stack, Bottom Tab Navigator, and Operational Stack) utilizing the React Navigation library.
- **Interface Technologies:** Expo-Linear-Gradient is used for visual continuity, and Expo-Font is utilized for customized typography (DM Sans and Playfair Display). Consistent experience across all devices is ensured via hardware-level "Safe Area" management.
- **Algorithm Layer:** In future phases, the machine learning-based FSRS (Free Spaced Repetition Scheduler) algorithm, which analyses the user's memory state, will be fully implemented.

4.3 Data Model

Syntagma's data model is structured to track the learning process:

- **User Data:** Secure JWT-based authentication data.
- **Content Structure:** Flashcards consist of a front side (word, example sentence, phonetic info) and an expanded back side (translation, voice pronunciation, visual context).
- **Statistical Data:** Metrics such as daily review counts, learned word volume, and accuracy rates are fetched via API and visualized on a dashboard.

4.4 Synchronization and Offline Strategy

A multi-layered strategy is followed for an uninterrupted user experience:

- **Offline Support:** To maintain functionality in environments without internet access (e.g., airplanes, subways), data is stored in Local Storage.

- Cloud Synchronization: Local data is automatically synchronized with the backend once an internet connection is established.
- Cross-Platform Integration: Instant synchronization of cards created via a Chrome extension with the mobile app is planned for future implementation.

4.5 Security

Application security and data integrity are protected through the following methods:

- Authentication: All login and registration processes are managed with a secure JWT (JSON Web Token) infrastructure.
- Data Transmission: All data flow between the client and server is carried out over industry-standard encryption protocols.
- Access Control: Operational layers and user-specific statistics are restricted within the framework of authorized sessions.

4.6 Testing and Quality

The application undergoes design and functional testing processes to maintain quality standards:

- Visual Quality: The consistency of the academic design system, featuring #F2EDE4 (Beige) and #6B4226 (Brown) tones, is tested across different screen resolutions.
- Performance: A low-latency experience is targeted through hardware-level Safe Area management and resource usage optimization.
- User Feedback: The accuracy of the FSRS algorithm is analyzed through the "Remembered/Forgot" cycle, alongside the impact of the smart notification system (fewer than 3 per day) on the user.

Section 5: Testing and Validation

1. Introduction

1.1 Version Control

Version No	Description of Changes	Date
1.0	First Syntagma version of the testing and validation document	May 15, 2026

1.2 Overview

This document describes the test plan, test design, and detailed test cases for the Syntagma application. Syntagma combines a Chrome extension, a Spring Boot backend, and a React Native mobile companion app to help language learners turn web pages and video content into contextual flashcards that are reviewed with a spaced repetition workflow.

Testing focuses on verifying that the implemented system can authenticate users, parse learning content, calculate comprehension, create and organize flashcards, capture subtitle/media context, schedule reviews with the FSRS algorithm, synchronize learning data.

1.3 Scope

The scope of this document covers functional validation for the current application version in the repository: backend API modules, Chrome extension workflows, and mobile review behavior. The plan includes expected pass/fail criteria, feature-level test design specifications, and executable-style manual/automated test cases.

1.4 Terminology

Acronym	Definition
AUTH	Authentication and Account Management
CPA	Content Parsing and Comprehension Analysis
FCC	Flashcard Creation and Collections
VSM	Video Subtitle and Media Capture
SRS	Spaced Repetition Review and Mobile Learning
FSRS	Free Spaced Repetition Scheduler
JWT	JSON Web Token
TC	Test Case

2. Features to be Tested

This section lists the major Syntagma features that will be validated. Each feature has a corresponding test design specification and detailed test cases later in the document.

2.1 Authentication and Account Management (AUTH)

Registration, login, JWT-protected routes, profile retrieval, and profile update behavior for authenticated Syntagma users.

2.2 Content Parsing and Comprehension Analysis (CPA)

Chrome extension extraction of page text, tokenization, lemmatization, known-word matching, and comprehension score calculation.

2.3 Flashcard Creation and Collections (FCC)

Word popup actions, AI-supported translation/example generation, manual card creation and editing, backend flashcard CRUD, and collection membership.

2.4 Video Subtitle and Media Capture (VSM)

Video-platform subtitle extraction, overlay display, fallback subtitle retrieval, screenshot capture, audio capture, and private media upload records.

2.5 Spaced Repetition Review and Mobile Learning (SRS)

FSRS scheduling, due-card retrieval, review submission, review logs, statistics, and mobile session feedback.

3. Features Not to be Tested

- Complex social functionality, and global leaderboard functionality is out of scope for the current MVP.
- Real-time conversational AI tutoring and voice-based roleplay are not part of this testing cycle.
- Full multi-language support beyond English target content and Turkish native-language assumptions is not included.
- External app-store deployment validation for iOS and Android is outside the scope of this document.
- Third-party platform availability, such as changes in YouTube caption infrastructure, is validated only through application fallback behavior.

4. Item Pass/Fail Criteria

A test case passes when the observed application behavior matches the expected behavior stated in the test procedure. A test case fails when the actual output, state transition, stored data, UI behavior, or security response differs from the predicted output.

4.1 Exit Criteria

Testing is considered successful when all planned high-priority test cases are executed and pass, at least 90% of planned test cases pass overall, and no critical defects remain in authentication, user-data ownership, flashcard creation, media persistence, or review scheduling.

5. References

No	Reference
[1]	Syntagma Product Vision & Scope Document, 2025-2026
[2]	Syntagma Implementation Details Document, 2025-2026
[3]	Syntagma CENG 408 Project Report, 2025-2026
[4]	Backend source code and tests under backend/src/main and backend/src/test
[5]	Chrome extension source code and tests under extension/src
[6]	Mobile application source code under mobile/src

6. Test Design Specifications

6.1 Authentication and Account Management (AUTH)

6.1.1 Subfeatures to be tested

Code	Subfeature
AUTH.REG	User registration
AUTH.LOG	User login
AUTH.JWT	JWT authorization enforcement
AUTH.PRO	Profile retrieval and update

6.1.2 Requirements

Req. ID	Requirement
R1	The system shall allow new users to register with validated credentials.
R2	The system shall issue a JWT after successful login and reject protected requests without a valid token.
R3	The system shall return and update only the authenticated user's profile data.

6.1.3 Test Cases

TC ID	Description
AUTH.REG.01	Verify that a new user can register with valid email, password, and profile information.
AUTH.LOG.01	Verify that an existing user can log in and receive a usable JWT.
AUTH.JWT.01	Verify that protected backend endpoints reject unauthenticated requests.
AUTH.PRO.01	Verify that the authenticated user's profile can be retrieved and updated.

6.2 Content Parsing and Comprehension Analysis (CPA)

6.2.1 Sub features to be tested

Code	Subfeature
CPA.TOK	DOM text extraction and token blocks
CPA.LEM	English token normalization and lemmatization
CPA.KNW	Known-word state matching
CPA.SCR	Comprehension and i+1 scoring
CPA.REA	Reader workspace analysis view

6.2.2 Requirements

Req. ID	Requirement
R4	The extension shall parse readable text without disturbing the original web page structure.
R5	The extension shall identify known, unknown, and ignored words using stored lexeme state.
R6	The system shall calculate a comprehension percentage and rank unknown words for learner action.

6.2.3 Test Cases

TC ID	Description
CPA.TOK.01	Verify that the extension extracts readable token blocks from supported page elements.
CPA.LEM.01	Verify that extracted English words are normalized into comparable lexical forms.
CPA.KNW.01	Verify that known, unknown, and ignored word states are applied to analyzed text.
CPA.SCR.01	Verify that the comprehension score and unknown-word ranking are calculated correctly.
CPA.REA.01	Verify that the reader workspace opens and displays analyzed text interactively.

6.3 Flashcard Creation and Collections (FCC)

6.3.1 Subfeatures to be tested

Code	Subfeature
FCC.POP	Word popup learning actions
FCC.AI	AI translation and example assistance
FCC.CRT	Flashcard creation workflow
FCC.EDT	Flashcard edit workflow
FCC.COL	Collection management
FCC.MAP	Backend-to-extension flashcard mapping

6.3.2 Requirements

Req. ID	Requirement
R7	The extension shall let users create contextual flashcards from selected words or consumed content.
R8	The backend shall store, list, update, and delete flashcards for the authenticated user.
R9	The backend shall group flashcards into collections and return collection identifiers to clients.
R10	AI endpoints shall return structured assistance for translation, word explanation, sentence explanation, and examples.

6.3.3 Test Cases

TC ID	Description
FCC.POP.01	Verify that clicking or selecting a word opens the word popup with learning actions.
FCC.CRT.01	Verify that the user can create a contextual flashcard from extension data.
FCC.EDT.01	Verify that an existing flashcard can be edited without losing attached metadata.
FCC.COL.01	Verify that flashcards can be grouped into collections.
FCC.MAP.01	Verify that backend flashcard responses are mapped correctly in the extension.

6.4 Video Subtitle and Media Capture (VSM)

6.4.1 Subfeatures to be tested

Code	Subfeature
VSM.YT	YouTube subtitle extraction
VSM.FB	Timed-text fallback retrieval
VSM.DSP	Subtitle overlay display
VSM.SS	Screenshot capture
VSM.AUD	Sentence audio capture
VSM.MEDIA	Media presign and retrieval

6.4.2 Requirements

Req. ID	Requirement
R11	The extension shall capture subtitles from supported video pages and convert them into synchronized cues.
R12	The extension shall support screenshot and audio assets when creating richer flashcards.
R13	The backend shall create private media records and provide presigned upload/download URLs.

6.4.3 Test Cases

TC ID	Description
VSM.YT.01	Verify that YouTube subtitle segments are extracted and converted into cue objects.
VSM.FB.01	Verify that subtitle capture falls back to timed-text retrieval when primary extraction returns no cues.
VSM.DSP.01	Verify that synchronized subtitle text is displayed in the video overlay.
VSM.SS.01	Verify that the extension can capture a screenshot frame for flashcard context.
VSM.AUD.01	Verify that sentence audio data can be attached to a flashcard payload.
VSM.MEDIA.01	Verify that media upload and retrieval use backend presigned URL records.

6.5 Spaced Repetition Review and Mobile Learning (SRS)

6.5.1 Subfeatures to be tested

Code	Subfeature
SRS.DUE	Due-card and daily-card retrieval
SRS.NEW	New-card FSRS scheduling
SRS.LAP	Forgotten-card relearning behavior
SRS.REV	Review submission and log persistence
SRS.STA	Review statistics
SRS.MOB	Mobile review session flow

6.5.2 Requirements

Req. ID	Requirement
R14	The system shall schedule flashcards with FSRS state values after each review.
R15	The mobile app shall support daily review sessions and display session outcomes.
R16	The backend shall expose review statistics including total, weekly, monthly, yearly, streak, and average result values.

6.5.3 Test Cases

TC ID	Description
SRS.DUE.01	Verify that due and daily card endpoints return cards scheduled for review.
SRS.NEW.01	Verify that a new card receives correct initial FSRS scheduling after a Good rating.
SRS.LAP.01	Verify that a forgotten review card enters relearning state and increments lapse count.
SRS.REV.01	Verify that review submission updates SRS state and creates a review log.
SRS.STA.01	Verify that review statistics are calculated for dashboard and mobile feedback.
SRS.MOB.01	Verify that the mobile app completes a review session and displays a summary.

7. Detailed Test Cases

7.1 AUTH.REG.01

TC_ID	AUTH.REG.01
Purpose	Verify that a new user can register with valid email, password, and profile information.
Requirements	R1
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Backend application and database are running.
Setup	Open the registration page or send a POST request to <code>/api/auth/register</code> with valid user data.
Procedure	[A01] Enter valid registration credentials. [A02] Submit the registration form or request. [V01] Verify a success response is returned. [V02] Verify the response contains authenticated user information and a token.
Cleanup	Delete the test user if the environment is shared.

7.2 AUTH.LOG.01

TC_ID	AUTH.LOG.01
Purpose	Verify that an existing user can log in and receive a usable JWT.
Requirements	R2
Priority	High
Estimated Time Needed	2 Minutes
Dependency	AUTH.REG.01 should pass or an existing test account must be available.
Setup	A registered user exists in the system.
Procedure	[A01] Submit valid credentials to <code>/api/auth/login</code> . [V01] Verify the response status is successful.

	[V02] Verify the JWT is present and can be attached to subsequent API requests.
Cleanup	None.

7.3 AUTH.JWT.01

TC_ID	AUTH.JWT.01
Purpose	Verify that protected backend endpoints reject unauthenticated requests.
Requirements	R2
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Security configuration must be active.
Setup	Prepare a request to a protected endpoint such as /api/flashcards without an Authorization header.
Procedure	[A01] Send the protected request without a token. [V01] Verify the request is rejected with an unauthorized or forbidden response. [A02] Repeat with a valid Bearer token. [V02] Verify the request is accepted.
Cleanup	None.

7.4 AUTH.PRO.01

TC_ID	AUTH.PRO.01
Purpose	Verify that the authenticated user's profile can be retrieved and updated.
Requirements	R3
Priority	Medium
Estimated Time Needed	4 Minutes
Dependency	AUTH.LOG.01 should pass.

Setup	Log in and attach the JWT to /api/users/me requests.
Procedure	[A01] Request the current user profile. [V01] Verify only the current user's profile is returned. [A02] Update editable profile fields. [V02] Verify the saved profile reflects the update.
Cleanup	Restore the original profile values if needed.

7.5 CPA.TOK.01

TC_ID	CPA.TOK.01
Purpose	Verify that the extension extracts readable token blocks from supported page elements.
Requirements	R4
Priority	High
Estimated Time Needed	4 Minutes
Dependency	The extension content script must be loaded on a page with readable English text.
Setup	Open a test page with paragraphs, headings, and unrelated navigation text.
Procedure	[A01] Start page analysis from the extension. [V01] Verify supported readable blocks are extracted. [V02] Verify hidden or unsupported page elements are not treated as learning content.
Cleanup	Close the test tab.

7.6 CPA.LEM.01

TC_ID	CPA.LEM.01
Purpose	Verify that extracted English words are normalized into comparable lexical forms.
Requirements	R5
Priority	High

Estimated Time Needed	3 Minutes
Dependency	Tokenization must produce word tokens.
Setup	Use a sample sentence containing inflected English words.
Procedure	[A01] Run the parser and lemmatizer on the sample content. [V01] Verify punctuation and non-word fragments are ignored. [V02] Verify inflected words can be matched against their stored lexical state.
Cleanup	None.

7.7 CPA.KNW.01

TC_ID	CPA.KNW.01
Purpose	Verify that known, unknown, and ignored word states are applied to analyzed text.
Requirements	R5
Priority	High
Estimated Time Needed	2 Minutes
Dependency	Extension storage contains lexeme state records.
Setup	Mark one word as known, one word as ignored, and leave another word unknown.
Procedure	[A01] Analyze text containing all three words. [V01] Verify known words are counted as understood. [V02] Verify ignored words are excluded from difficulty calculations. [V03] Verify unknown words remain visible for learning actions.
Cleanup	Remove temporary lexeme state records.

7.8 CPA.SCR.01

TC_ID	CPA.SCR.01
Purpose	Verify that the comprehension score and unknown-word ranking are calculated correctly.

Requirements	R6
Priority	High
Estimated Time Needed	3 Minutes
Dependency	CPA.KNW.01 should pass.
Setup	Prepare a text sample with a known expected ratio of understood to unknown words.
Procedure	[A01] Run analysis on the sample. [V01] Compare the displayed comprehension percentage with the expected value. [V02] Verify the unknown-word list is ordered by relevance or frequency.
Cleanup	None.

7.9 CPA.REA.01

TC_ID	CPA.REA.01
Purpose	Verify that the reader workspace opens and displays analyzed text interactively.
Requirements	R4, R6
Priority	Medium
Estimated Time Needed	5 Minutes
Dependency	Reader page build must be available.
Setup	Open the extension reader workspace with a sample text source.
Procedure	[A01] Load the reader page. [A02] Start analysis. [V01] Verify text segments are displayed. [V02] Verify word-level interactions are available from the reader view.
Cleanup	Close the reader page.

7.10 FCC.POP.01

TC_ID	FCC.POP.01
Purpose	Verify that clicking or selecting a word opens the word popup with learning actions.
Requirements	R7
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Content overlay must be mounted on the page.
Setup	Open an analyzed page with unknown words.
Procedure	[A01] Click an unknown word. [V01] Verify the popup appears near the selected word. [V02] Verify dictionary, status, and create-card actions are visible and usable.
Cleanup	Dismiss the popup.

7.11 FCC.AI.01

TC_ID	FCC.AI.01
Purpose	Verify that AI endpoints return structured learning assistance.
Requirements	R10
Priority	Medium
Estimated Time Needed	3 Minutes
Dependency	Backend AI configuration is available or a test double is configured.
Setup	Prepare valid requests for translation, word explanation, sentence explanation, and example generation.
Procedure	[A01] Submit each AI request through /api/ai. [V01] Verify each response follows the expected response model. [V02] Verify empty or invalid input is handled without crashing the backend.
Cleanup	None.

7.12 FCC.CRT.01

TC_ID	FCC.CRT.01
Purpose	Verify that the user can create a contextual flashcard from extension data.
Requirements	R7, R8
Priority	High
Estimated Time Needed	6 Minutes
Dependency	AUTH.LOG.01 and FCC.POP.01 should pass.
Setup	Select a word from page or video context and open the card creator.
Procedure	[A01] Fill front, back, sentence, and optional media fields. [A02] Save the flashcard. [V01] Verify the backend returns the created card. [V02] Verify the card appears in the extension or options workspace list.
Cleanup	Delete the created flashcard if needed.

7.13 FCC.EDT.01

TC_ID	FCC.EDT.01
Purpose	Verify that an existing flashcard can be edited without losing attached metadata.
Requirements	R8
Priority	Medium
Estimated Time Needed	5 Minutes
Dependency	FCC.CRT.01 should pass.
Setup	Open an existing flashcard in edit mode.
Procedure	[A01] Change the card back text and one optional field. [A02] Save the update. [V01] Verify the changed fields are persisted. [V02] Verify unchanged fields such as collection membership remain intact.
Cleanup	Restore original card data or delete the card.

7.14 FCC.COL.01

TC_ID	FCC.COL.01
Purpose	Verify that flashcards can be grouped into collections.
Requirements	R9
Priority	High
Estimated Time Needed	6 Minutes
Dependency	Authenticated backend access and at least one flashcard are available.
Setup	Create or select a flashcard and create a collection.
Procedure	[A01] Add the flashcard to the collection. [V01] Verify the collection item is created. [A02] Request the collection details. [V02] Verify the flashcard appears in the collection response.
Cleanup	Remove the collection item and test collection.

7.15 FCC.MAP.01

TC_ID	FCC.MAP.01
Purpose	Verify that backend flashcard responses are mapped correctly in the extension.
Requirements	R8, R9
Priority	Medium
Estimated Time Needed	4 Minutes
Dependency	Backend flashcard list endpoint returns at least one card with collectionIds.
Setup	Open the extension workspace that lists flashcards.
Procedure	[A01] Load flashcards from the backend. [V01] Verify front, back, sentence, media fields, and collectionIds are mapped into the extension payload. [V02] Verify cards can be filtered or displayed by collection membership.

Cleanup	None.
----------------	-------

7.16 VSM.YT.01

TC_ID	VSM.YT.01
Purpose	Verify that YouTube subtitle segments are extracted and converted into cue objects.
Requirements	R11
Priority	High
Estimated Time Needed	5 Minutes
Dependency	A YouTube watch page with English captions is available.
Setup	Open a supported YouTube video and enable the extension video subsystem.
Procedure	[A01] Request subtitle capture for English. [V01] Verify cue text, start time, end time, and index are populated. [V02] Verify the cue order matches the video timeline.
Cleanup	Close the video tab.

7.17 VSM.FB.01

TC_ID	VSM.FB.01
Purpose	Verify that subtitle capture falls back to timed-text retrieval when primary extraction returns no cues.
Requirements	R11
Priority	Medium
Estimated Time Needed	5 Minutes
Dependency	Primary extractor failure can be simulated.
Setup	Use a video page or mocked response where initial extraction returns an empty segment list.

Procedure	[A01] Start subtitle capture. [V01] Verify the fallback request is attempted. [V02] Verify returned fallback cues are converted into the same cue model.
Cleanup	Restore normal extractor behavior.

7.18 VSM.DSP.01

TC_ID	VSM.DSP.01
Purpose	Verify that synchronized subtitle text is displayed in the video overlay.
Requirements	R11
Priority	High
Estimated Time Needed	5 Minutes
Dependency	VSM.YT.01 should pass.
Setup	Load a video with captured subtitle cues.
Procedure	[A01] Play the video. [V01] Verify the overlay displays the current subtitle cue. [A02] Seek forward or backward. [V02] Verify the displayed cue updates according to playback time.
Cleanup	Disable the overlay.

7.19 VSM.SS.01

TC_ID	VSM.SS.01
Purpose	Verify that the extension can capture a screenshot frame for flashcard context.
Requirements	R12
Priority	Medium
Estimated Time Needed	4 Minutes
Dependency	A playable video element is available.

Setup	Pause a video at a visually identifiable frame.
Procedure	[A01] Trigger screenshot capture. [V01] Verify a JPEG data URL is produced. [V02] Verify fallback capture behavior is used if intrinsic video dimensions are unavailable.
Cleanup	Remove the temporary screenshot from draft data.

7.20 VSM.AUD.01

TC_ID	VSM.AUD.01
Purpose	Verify that sentence audio data can be attached to a flashcard payload.
Requirements	R12
Priority	Medium
Estimated Time Needed	4 Minutes
Dependency	Browser media recording support is available.
Setup	Open the video card creation workflow.
Procedure	[A01] Record or attach a sentence audio clip. [V01] Verify the audio data URL is accepted by the payload model. [V02] Verify generated file names include flashcard context and a supported extension.
Cleanup	Discard the temporary recording if not saved.

7.21 VSM.MEDIA.01

TC_ID	VSM.MEDIA.01
Purpose	Verify that media upload and retrieval use backend presigned URL records.
Requirements	R13
Priority	High

Estimated Time Needed	7 Minutes
Dependency	S3-compatible storage configuration is available.
Setup	Create a flashcard and prepare a small image or audio file.
Procedure	[A01] Request a media presign URL. [V01] Verify uploadUrl, objectKey, and expiration are returned. [A02] Create a media record after upload. [A03] Request a download URL. [V02] Verify the URL is returned only for authorized access.
Cleanup	Delete the media record.

7.22 SRS.DUE.01

TC_ID	SRS.DUE.01
Purpose	Verify that due and daily card endpoints return cards scheduled for review.
Requirements	R14
Priority	High
Estimated Time Needed	5 Minutes
Dependency	At least one flashcard with SRS state exists.
Setup	Set one card as due and one card as not due.
Procedure	[A01] Request /api/srs/due. [V01] Verify due cards are returned. [A02] Request /api/srs/daily. [V02] Verify the daily response includes the expected review candidates.
Cleanup	Restore original schedule values if manually changed.

7.23 SRS.NEW.01

TC_ID	SRS.NEW.01
Purpose	Verify that a new card receives correct initial FSRS scheduling after a Good or Easy rating.

Requirements	R14
Priority	High
Estimated Time Needed	4 Minutes
Dependency	FSRS algorithm service is available.
Setup	Create a new SRS state for a flashcard.
Procedure	[A01] Submit a Good rating. [V01] Verify the card enters learning state with a short next-review interval. [A02] Submit an Easy rating for a new state. [V02] Verify the card can graduate directly to review state.
Cleanup	Reset or remove the test SRS state.

7.24 SRS.LAP.01

TC_ID	SRS.LAP.01
Purpose	Verify that a forgotten review card enters relearning state and increments lapse count.
Requirements	R14
Priority	High
Estimated Time Needed	4 Minutes
Dependency	An existing review-state card is available.
Setup	Prepare a card with REVIEW state and a prior review date.
Procedure	[A01] Submit an Again rating. [V01] Verify the card state becomes RELEARNING. [V02] Verify lapse count increments and the next review is scheduled soon.
Cleanup	Remove the test card or restore its state.

7.25 SRS.REV.01

TC_ID	SRS.REV.01
Purpose	Verify that review submission updates SRS state and creates a review log.
Requirements	R14, R16
Priority	High
Estimated Time Needed	5 Minutes
Dependency	Authenticated user owns the target flashcard.
Setup	Open a review request for a user-owned flashcard.
Procedure	[A01] Submit a review result from mobile or API. [V01] Verify a review log is saved. [V02] Verify the returned SRS state reflects the rating and next review schedule.
Cleanup	Delete temporary review data if required.

7.26 SRS.STA.01

TC_ID	SRS.STA.01
Purpose	Verify that review statistics are calculated for dashboard and mobile feedback.
Requirements	R16
Priority	Medium
Estimated Time Needed	4 Minutes
Dependency	Review logs exist for the authenticated user.
Setup	Create or select a user with weekly, monthly, and yearly review logs.
Procedure	[A01] Request /api/reviews/stats. [V01] Verify total, weekly, monthly, yearly, streak, and average result values are returned. [V02] Verify a user with no logs receives sensible zero/default values.

Cleanup	None.
----------------	-------

7.27 SRS.MOB.01

TC_ID	SRS.MOB.01
Purpose	Verify that the mobile app completes a review session and displays a summary.
Requirements	R15
Priority	High
Estimated Time Needed	8 Minutes
Dependency	Mobile app can reach the backend or a mocked API layer.
Setup	Log in on the mobile app and ensure due flashcards are available.
Procedure	[A01] Start a review session from the mobile home or review screen. [A02] Answer cards using available rating controls. [V01] Verify each answer advances the session. [V02] Verify the session summary shows completed count and progress feedback.
Cleanup	Log out of the test account if using a shared device.

Section 6: Conclusion and Future Work

1. Project Summary

Syntagma is an AI-powered language learning application designed to help Turkish-speaking learners acquire English vocabulary through spaced repetition, contextual learning, and AI-driven explanations. The project encompasses three major components: a Spring Boot backend REST API, a React Native mobile application, and a Chrome browser extension, all working together to deliver a cohesive, multi-platform learning experience.

The application targets a real gap in existing language learning tools: the lack of an integrated system that combines scientifically backed spaced repetition scheduling with in-context vocabulary acquisition from real-world content such as web pages, videos, and eBooks.

2. Key Achievements

2.1 FSRS-Based Spaced Repetition Engine

The core of Syntagma is its implementation of the Free Spaced Repetition Scheduler [1] (FSRS) algorithm, a modern, research-backed alternative to the classic SM-2 [2] algorithm. The backend SRS engine manages card states (stability, difficulty, reps, lapses), computes optimal review intervals, and serves daily review queues that mix due cards with new cards. The system automatically promotes words to "KNOWN" status after sustained recall success (≥ 3 successful reviews with a scheduled interval exceeding 25 days), ensuring that mastered vocabulary exits the review cycle organically.

2.2 AI-Powered Language Assistance

A dedicated AI service integrates with the OpenRouter API [3] to provide three distinct AI-powered features, all tailored for Turkish-speaking learners:

Word Explanation: Given a word and its surrounding sentence, the AI returns the Turkish meaning, part of speech, usage notes, common mistakes, and example sentences — all in Turkish.

Sentence Translation: Provides natural, literal, and alternative Turkish translations for English sentences, helping learners understand nuance and idiomatic usage.

Sentence Grammar Explanation: Breaks down sentence structure into grammatical chunks, explains tense/aspect/voice in Turkish, and offers concrete learner tips.

All AI calls are routed through the backend (/api/ai endpoints), keeping API keys secure and enabling centralized rate limiting and monitoring.

2.3 Chrome Extension Learning Features

The Syntagma Chrome extension (Manifest V3) is a comprehensive learning companion that operates directly within the learner's browsing environment. Key capabilities include:

In-Page Word Popup: Users can click or select any word on a web page to see its definition, AI explanation, and instantly create a flashcard — all without leaving the page.

EPUB eBook Reader: A built-in reader allows users to upload and read EPUB files within the extension. Words can be looked up and turned into flashcards in context, with reading progress tracked and synced to the backend.

Video Learning with Audio/Screenshot Capture: The extension captures sentence-level audio and screenshots from videos (e.g., YouTube subtitles), enabling multimedia flashcard creation with real audio pronunciations and visual context.

Page Comprehension Analysis: Analyzes page text against the user's known vocabulary to estimate comprehension percentage, helping learners choose appropriate reading material.

Known Words Management: Users can view, filter, and manage their known words list. CEFR-level intake allows bulk-marking words as known based on proficiency level (A1–C2).

Manual Flashcard Creator: A dedicated card creator page allows users to build flashcards manually with custom front/back content, audio, and images.

2.4 Mobile Application

The React Native (Expo) mobile app provides on-the-go access to the learning system with the following screens and features:

Flashcard Review: Interactive review sessions using the SRS scheduling grading and session summaries showing performance metrics.

Flashcard Library: Browse, search, and manage the complete flashcard collection with filtering by knowledge status.

Badge System: Achievement badges reward milestones (cards reviewed, streaks, words mastered), increasing user motivation and engagement.

Push Notifications: Daily review reminders via expo-notifications keep learners on track with their spaced repetition schedule.

Settings & Language Management: Users can configure daily card limits, manage target languages, and customize their learning experience.

2.5 Robust Backend Architecture

The Spring Boot backend, built with Java 17 and backed by PostgreSQL, provides a comprehensive REST API with the following architectural highlights:

JWT Authentication & Security: Token-based authentication with Spring Security, including email verification and secure password handling.

Cloud Storage Integration: S3-compatible object storage (DigitalOcean Spaces) for eBook files and media assets, with pre-signed URL generation for secure uploads and downloads.

Offline-First Synchronization: A dedicated sync service (SyncController/SyncService) enables both mobile and extension clients to push local changes and pull server updates, supporting offline usage with eventual consistency.

Collection Management: Flashcards can be organized into user-defined collections, with full CRUD operations and item management.

Media Asset Management: Audio recordings and screenshot images are stored as MediaAsset entities linked to flashcards, supporting rich multimedia cards.

Review Analytics: Review logs track every review event with timestamps, device info, and results, enabling detailed learning analytics and progress reporting.

OpenAPI Documentation: Springdoc integration provides auto-generated Swagger UI for API exploration and testing.

2.6 Word Knowledge Tracking

The WordKnowledge system maintains a per-user vocabulary profile with four statuses: KNOWN, LEARNING, UNKNOWN, and IGNORED. The system supports batch updates, CEFR-level intake (automatically marking all words at or below a given proficiency level as known), and bidirectional integration with the flashcard review cycle. Words that are mastered through spaced repetition are automatically promoted to KNOWN status.

3. Limitations

Single Language Pair: The current implementation is specifically designed for Turkish speakers learning English. The AI prompts, CEFR word lists, and UI text are hardcoded for this pair. Supporting additional language combinations would require significant refactoring of the AI prompt templates and vocabulary databases.

AI API Dependency: The AI features depend on the OpenRouter [3] API (external third-party service). This introduces latency, cost per request, and a single point of failure. There is no fallback mechanism if the AI service is unavailable.

Limited Automated Testing: While some unit tests exist (audio recorder, screenshot capture, flashcard logic, page analysis), the overall test coverage -particularly for integration tests and end-to-end flows - is limited. The backend uses H2 for test environments but lacks comprehensive API-level test suites.

Extension Platform Lock-In: The browser extension is built exclusively for Chrome (Manifest V3). Firefox, Safari, and other browsers are not supported.

Scalability Concerns: The current architecture uses a single PostgreSQL instance with no caching layer (e.g., Redis). Under high user load, the AI proxy endpoints and SRS computation could become bottlenecks.

4. Lessons Learned

AI Prompt Engineering Is Iterative: Designing AI prompts that consistently return valid JSON with Turkish-language content required multiple iterations. Early versions produced inconsistent formatting, mixed-language responses, or hallucinated grammar terms. Structured JSON schemas in the system prompt and explicit language requirements significantly improved reliability.

Multi-Platform Development Increases Complexity Exponentially: Maintaining three client platforms (backend, mobile, extension) with shared data models but different runtime environments (JVM, React

Native [5], Chrome extension sandbox) created significant coordination overhead. Schema changes required updates across all three codebases.

Chrome Extension Manifest V3 Constraints: Migrating to Manifest V3 introduced challenges with service worker lifecycle management (no persistent background pages), storage limitations, and content script injection timing. The service worker's ephemeral nature required careful state management using Chrome Storage API and IndexedDB.

Offline-First Is Harder Than Expected: Building reliable offline support with eventual sync required rethinking data flow. Edge cases — such as creating a flashcard offline, reviewing it before syncing, then syncing with a server that assigns different IDs — demanded careful handling of local vs. server identity.

5. Conclusion, Shortcomings, Future Works

Syntagma demonstrates that integrating spaced repetition science with modern AI capabilities and multi-platform delivery can create a powerful, context-aware language learning tool. By embedding vocabulary acquisition directly into the learner's natural browsing and reading activities rather than confining it to a standalone app, the system lowers the barrier to consistent practice.

The FSRS algorithm ensures that review scheduling is both efficient and personalized, while the AI-powered explanations provide the kind of contextual, native-language support that traditional flashcard apps lack. The combination of a Chrome extension for in-context learning, a mobile app for on-the-go review, and a robust backend for synchronization creates a genuinely multi-platform learning ecosystem.

While limitations remain particularly around language pair flexibility, test coverage, and scalability the foundation is solid and extensible. The lessons learned during development, especially around AI prompt engineering, and cross-platform data synchronization, provide valuable guidance for the future evolution of the platform. Syntagma stands as a functional prototype that validates the core thesis: AI-enhanced spaced repetition, delivered in context, can meaningfully improve the vocabulary acquisition process for language learners.

5.1. Backend Application

5.1.1 Conclusion

The Spring Boot backend was successfully implemented, providing JWT-based authentication, FSRS spaced repetition scheduling, flashcard and collection management, eBook storage via S3-compatible presigned URLs, word knowledge tracking, and AI-powered language assistance through the OpenRouter API. The REST API serves both the Chrome extension and mobile application with a unified data model backed by PostgreSQL.

5.1.2 Shortcomings

Conflict resolution for offline sync scenarios where multiple devices edit the same flashcard simultaneously is not fully addressed.

5.1.3 Future Works

Planned improvements include a Redis caching layer for performance optimization, improving the sync conflict resolution strategy for multi-device scenarios and adding application-level rate limiting and usage analytics for the AI features.

5.2 Chrome Extension

5.2.1 Conclusion

The Chrome extension (Manifest V3) was successfully implemented as the learning interface, providing in-page word interaction with popup definitions, CEFR-based vocabulary classification, a video subsystem with subtitle capture and audio recording, an eBook reader, a flashcard creation and editing workspace, settings management, and English and Turkish user interfaces. The extension operates with a service-worker background architecture and communicates with the backend API for data persistence and AI features.

5.2.2 Shortcomings

The tokenizer and lemmatizer, while enhanced with contraction expansion and morphological rules, are rule-based and may misclassify irregular or domain-specific vocabulary. The dictionary.json file must be manually placed in the workspace directory. Subtitle capture depends on platform-specific DOM structures (e.g., YouTube) which may break when those sites update their layout. The video audio capture via `captureStream()` is not available on DRM-protected content.

5.2.3 Future Works

Replacing the rule-based lemmatizer with a more robust NLP model, adding support for additional subtitle sources and streaming platforms, implementing a content recommendation engine that suggests reading and video material matched to the user's proficiency level, adding text-to-speech integration for flashcard pronunciation, and expanding the i18n system to support additional interface languages beyond English and Turkish.

5.3 Mobile Application

5.3.1 Conclusion

The React Native mobile application was successfully implemented as the on-the-go learning interface of Syntagma, providing users with SRS-scheduled flashcard review sessions, a gamified badge system, and real-time progress tracking through weekly and monthly statistics charts. By integrating push notifications with configurable daily reminders and streak warnings, the app ensures consistent user engagement beyond desktop environments.

5.3.2 Shortcomings

The mobile application does not include an offline mode; all core functionalities including flashcard review, vocabulary tracking, and statistics requires an active internet connection, which may interrupt the learning experience in low-connectivity environments.

5.3.3 Future Works

A planned improvement is the introduction of offline review support with background synchronization, allowing users to complete study sessions without an active connection and sync results when connectivity is restored. Expanding the analytics screen with per-collection retention curves and predicted vocabulary mastery dates would further enrich the learning insights available on mobile.

5.4 Whole System

5.4.1 Conclusion

Syntaxma as a complete system successfully demonstrates the integration of spaced repetition science, AI-powered language assistance, and multi-platform delivery into a cohesive language learning tool. The three subsystems, backend API, Chrome extension, and mobile application, work together through a unified data model and synchronization layer to provide Turkish-speaking learners with a comprehensive English vocabulary acquisition experience across web browsing, video watching, eBook reading, and on-the-go review.

5.4.2 Shortcomings

The system is currently limited to a single language pair (English→Turkish). Cross-platform synchronization, while functional, does not handle all conflict scenarios.

5.4.3 Future Works

Future development should prioritize multi-language pair support to serve a broader audience, comprehensive end-to-end testing across all subsystems.

References

- [1] Duolingo, “Main Page,” [Online]. Available: <https://tr.duolingo.com/>. [Accessed 4 Nov 2025].
- [2] Memrise, “Memrise : Learn a Language,” [Online]. Available: <https://www.memrise.com>. [Accessed 11 Nov 2025].
- [3] Babbel, “Babbel – Language Learning,” [Online]. Available: <https://www.babbel.com>. [Accessed 11 Nov 2025].
- [4] Busuu, “Busuu: Learn Languages,” [Online]. Available: <https://www.busuu.com>. [Accessed 11 Nov 2025].
- [5] Rosetta Stone Ltd., “Rosetta Stone: Language Learning,” [Online]. Available: <https://www.rosettastone.com>. [Accessed 11 Nov 2025].

- [6] LingQ Inc., “LingQ – Language Learning App,” [Online]. Available: <https://www.lingq.com>. [Accessed 7 Nov 2025].
- [7] Anki, “What spaced repetition algorithm does Anki use?,” 2023. [Online]. Available: <https://faqs.ankiweb.net/what-spaced-repetition-algorithm.html>. [Accessed 4 Nov 2025].
- [8] SuperMemo Guru, “Three component model of memory,” [Online]. Available: https://supermemo.guru/wiki/Three-component_model_of_memory. [Accessed 27 Nov 2025].
- [9] Gradle, “Gradle User Manual,” [Online]. Available: <https://docs.gradle.org/current/userguide/userguide.html>. [Accessed 4 Nov 2025].
- [10] Hibernate, “Introduction to Hibernate ORM,” [Online]. Available: https://docs.hibernate.org/orm/7.1/introduction/html_single/. [Accessed 4 Nov 2025].
- [11] Amazon Web Services, “What Is Boilerplate Code?,” [Online]. Available: <https://aws.amazon.com/what-is/boilerplate-code/>. [Accessed 27 Nov 2025].
- [12] Baeldung, “Introduction to Project Lombok,” [Online]. Available: <https://www.baeldung.com/intro-to-project-lombok>. [Accessed 4 Nov 2025].
- [13] GeeksforGeeks, “Software Engineering | Mock Introduction,” [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/software-engineering-mock-introduction/>. [Accessed 27 Nov 2025].
- [14] Baeldung, “Mockito Annotations,” [Online]. Available: <https://www.baeldung.com/mockito-annotations>. [Accessed 4 Nov 2025].
- [15] C. A. H. A. a. M. P.-G. F. J. Palacios-Hidalgo, “Origins, concept and didactic applications – systematic review of literature (2012–2019),” *Technology, Knowledge and Learning*, vol. 25, no. 4, p. 853–879.
- [16] R. D. a. K. K. K. Guntupally, “Spring Boot based REST API to improve data quality report generation for big scientific data: ARM Data Center example,” in *Proc. IEEE Int. Conf. Big Data*, Seattle, WA, USA, 2018.
- [17] Wikipedia, “Facade pattern,” [Online]. Available: https://en.wikipedia.org/wiki/Facade_pattern. [Accessed 27 Nov 2025].
- [18] QOS.ch, “Name of Web Page:,” [Online]. Available: <https://www.slf4j.org/manual.html>. [Accessed 27 Nov 2025].
- [19] Wikipedia, “Coupling (computer programming),” [Online]. Available: [https://en.wikipedia.org/wiki/Coupling_\(computer_programming\)](https://en.wikipedia.org/wiki/Coupling_(computer_programming)). [Accessed 2025 Nov 2025].

- [20] QOS.ch, “Logback Manual,” [Online]. Available: <https://logback.qos.ch/manual/index.html>. [Accessed 27 Nov 2025].
- [21] M. N. Jeyaraj, S. Sucharitharathna, C. Senarath, Y. Kanagaraj and I. Udayakumara, “Cognitive visual-learning environment for PostgreSQL,” arXiv, 2022.
- [22] S. V. Salunke and A. Ouda, “A performance benchmark for the PostgreSQL and MySQL databases,” *Future Internet*, vol. 16, no. 10, p. 382, 2024.
- [23] DBeaver Community, “DBeaver: Free Universal Database Tool,” [Online]. Available: <https://dbeaver.io>. [Accessed 27 Nov 2025].
- [24] A. L. Mhatre, “Microservices architecture using Docker and Kubernetes,” *Int. J. for Multidisciplinary Research*, vol. 5, no. 5, p. 1–8, 2023.
- [25] A. Lama, “Documenting APIs with Swagger: Enhancing Developer Experience,” *International Journal of Science and Technology*, vol. 15, no. 2, pp. 165-169, 2025.
- [26] P. P. Kore, M. J. Lohar, M. T. Surve and S. Jadhav, “API testing using Postman tool,” *Int. J. Research in Applied Science and Engineering Technology*, vol. 10, no. 12, p. 841–843, 2022.
- [27] y.-d. Developers, “yt-dlp: A Feature-Rich Video Downloader (Fork of youtube-dl),” GitHub, 2021. [Online]. Available: <https://github.com/yt-dlp/yt-dlp>. [Accessed 11 Nov 2025].
- [28] I. Bernat, “netflix-to-srt: Export Netflix subtitles to SRT format,” GitHub Repository, 2019. [Online]. Available: <https://github.com/isaacbernat/netflix-to-srt>. [Accessed 7 Nov 2025].
- [29] Mozilla, “Readability,” Github, [Online]. Available: <https://github.com/mozilla/readability>. [Accessed 7 11 2025].
- [30] Graype Systems, “WinkNLP – Developer Friendly NLP in Node.js,” WinkJS Documentation, 2023. [Online]. Available: <https://winkjs.org>. [Accessed 11 Nov 2025].
- [31] S. K. Small, “compromise.js,” Github, 2015. [Online]. Available: <https://github.com/spencermountain/compromise>.
- [32] B. N. M. Brysbaert, “SUBTLEXus: Word frequencies based on 51 million words from American English television subtitles,” *Behavior Research Methods*, vol. 41, no. 3, p. 978–990, 2009.
- [33] I. M. S. V. L. a. A. B. M. Honnibal, “spaCy: Industrial-strength Natural Language Processing in Python,” 2020. [Online]. Available: <https://spacy.io>. [Accessed 7 Nov 2025].
- [34] Google, “Text-to-Speech API,” Google, [Online]. Available: <https://cloud.google.com/text-to-speech>. [Accessed 11 Nov 2025].
- [35] X. Z. a. Y. G. D. He, “Mobile-assisted language learning: A review of recent developments,” *Journal of Information Technology & Software Engineering*, vol. 9, no. 2, p. 1–6, 2019.

- [36] M. W. Y. Reinders, “The effects of gamification on language learning: A study using a mobile language learning app,” *Computer Assisted Language Learning*, vol. 36, no. 2, p. 195–216, 2023.
- [37] R. Gafni, D. B. Achituv and G. J. Rachmani, “Learning foreign languages using mobile applications,” *Journal of Information Technology Education: Research*, vol. 16, p. 301–317, 2017.
- [38] R. B. a. J. Nielsen, “User Interface Design for Mobile Applications,” *2019 IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC)*, 2019.
- [39] O. Viberg and Å. Grönlund, “Mobile assisted language learning: A literature review,” *11th World Conf. Mobile and Contextual Learning (mLearn 2012)*, vol. 955, p. 9–16, Oct 2012.
- [40] A. Wohllebe, D.-S. Hübner, U. Radtke and S. Podrutzik, “Mobile Apps in Retail: Effect of Push Notification Frequency on App User Behavior,” *Innovative Marketing*, vol. 17, no. 2, p. 102–111, 2021.
- [41] B. Eisenman, *Learning React Native: Building Native Mobile Apps with JavaScript*, Sebastopol, CA, USA: O’Reilly Media, 2016.
- [42] S. Contributors, “Spring Boot Reference Documentation,” Spring, [Online]. Available: <https://docs.spring.io/spring-boot/index.html>. [Accessed 10 April 2026].
- [43] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL, [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed 10 April 2026].
- [44] S. Contributors, “Jwt (Spring Security API),” Spring, [Online]. Available: <https://docs.spring.io/spring-security/site/docs/current/api/org/springframework/security/oauth2/jwt/Jwt.html>. [Accessed 10 April 2026].
- [45] SmartBear Software, “OpenAPI Specification,” Swagger, [Online]. Available: https://swagger.io/docs/specification/v3_0/about/. [Accessed 10 April 2026].
- [46] I. Postman, “Postman Introduction,” Postman Learning Center, [Online]. Available: <https://learning.postman.com/docs/introduction/overview>. [Accessed 10 April 2026].
- [47] Gradle Inc., “Gradle User Manual,” Gradle, [Online]. Available: <https://docs.gradle.org/current/userguide/userguide.html>. [Accessed 10 April 2026].
- [48] Red Hat, Inc., “Hibernate ORM User Guide,” Hibernate, [Online]. Available: https://docs.hibernate.org/orm/current/userguide/html_single/. [Accessed 10 April 2026].
- [49] o.-s.-r. Contributors, “fsrs4anki: A modern Spaced Repetition algorithm,” GitHub, [Online]. Available: <https://github.com/open-spaced-repetition/fsrs4anki>. [Accessed 10 April 2026].

- [50] open-spaced-repetition Contributors, “rs-fsrs-java: Java bindings for rs-fsrs,” GitHub, [Online]. Available: GitHub. <https://github.com/open-spaced-repetition/rs-fsrs-java>. [Accessed 10 April 2026].
- [51] Microsoft, “TypeScript: JavaScript With Syntax For Types,” [Online]. Available: <https://www.typescriptlang.org/>. [Accessed 10 April 2026].
- [52] Meta Platforms, Inc., “React: The library for web and native user interfaces,” React, [Online]. Available: <https://react.dev/>. [Accessed 10 4 2026].
- [53] E. Y. a. V. Contributors., “Vite: Next Generation Frontend Tooling,” Vite, [Online]. Available: <https://vite.dev/>. [Accessed 10 April 2026].
- [54] Google, “Manifest V3,” Chrome Developers, [Online]. Available: <https://developer.chrome.com/docs/extensions/mv3/>. [Accessed 10 April 2026].
- [55] V. Contributors, “Vitest: A blazing fast unit test framework powered by Vite,” Vitest, [Online]. Available: <https://vitest.dev/>. [Accessed 10 April 2026].
- [56] S. Kelly, “compromise: modest natural-language processing,” GitHub, [Online]. Available: <https://github.com/spencermountain/compromise>. [Accessed 10 April 2026].
- [57] uuid Contributors, “uuid,” npm, [Online]. Available: <https://www.npmjs.com/package/uuid>. [Accessed 10 April 2026].
- [58] Pieroxy, “lz-string,” npm, [Online]. Available: <https://www.npmjs.com/package/lz-string>. [Accessed 10 4 2026].
- [59] r. Contributors, “react-markdown,” npm, [Online]. Available: <https://www.npmjs.com/package/react-markdown>. [Accessed 10 April 2026].
- [60] OpenRouter, “Quickstart,” OpenRouter Documentation, [Online]. Available: <https://openrouter.ai/docs/quickstart>.
- [61] I. Meta Platforms, “React Native: A framework for building native apps using React,” React Native, [Online]. Available: <https://reactnative.dev/>. [Accessed 10 April 2026].
- [62] Expo, “Expo Documentation,” Expo, [Online]. Available: <https://docs.expo.dev/>. [Accessed 10 April 2026].